KairosSNNI Architectural Framework

The KairosSNNI architecture is organized into three primary layers: the Computational Infrastructure, the Secure Inference Service Engine (SISE), and the Applications Layer. Together, these components manage the complex resource and timing requirements of secure inference.

1. Computational Infrastructure (The Hardware and Policy Base)

This layer provides the physical resources and the static rules used for system operation.

- **System Resources:** This includes the raw hardware pool, specifically the *CPU cores (P)* and shared *memory (M)*. It also manages the high-speed storage for *Preprocessed files* and the available network *bandwidth*.
- **Model Catalog:** A centralized registry of all supported neural network models. Models are categorized by size and complexity into three classes: Small (e.g., M1, M2), Medium (e.g., HiNet, AlexNet), and Large (e.g., VGG-16).
- **Execution Parameters:** This acts as the system's rulebook. It defines how data and task parallelism are handled, the Service Level Objective (SLO) safety factors, the specific scheduling logic to be used, and the rules for how client requests are grouped into batches.

2. Secure Inference Service Engine (The Intelligence Layer)

The SISE is the "brain" of the framework. it is a middleware engine that coordinates the end-to-end execution of requests.

- **Admission Control:** The entry point for the server. It receives incoming requests and places valid ones into the Global Request Queue. It is responsible for filtering out impractical or impossible requests before they consume system time.
- **Profiler:** The predictive engine of the system. It uses an *Execution Time Predictor* and a *Resource Predictor* to estimate the "cost" of a job based on the model type and the slowness of the client.
- **Request Scheduler:** The central hub for resource decisions. It contains:
    - **Offline Scheduler:** A background "replenisher" that manages the pre-generation of cryptographic material. It prepares J_pre files in advance based on quotas defined for every model.
    - **Online Scheduler:** An optimizer that uses advanced logic (SA/DQN) to sort the final Priority Queue.
    - **Preprocessed File Manager:** Tracks the inventory of ready-to-use cryptographic files.
    - **Dispatcher:** The action unit that performs a "Triple Check" (cores, memory, and files) and pins jobs to physical CPU cores for isolated execution.
- **Bi-Phased SNNI Protocol (SHARK):** The cryptographic execution engine. It processes the work in two distinct phases: the Offline Preprocessing Phase (generating material) and the Online Inference Phase (calculating the prediction).
- **Resource Manager:** The monitor that tracks exactly how many cores and how much memory are being used at any given millisecond.

3. Applications Layer (The Multi-Tenant Environment)

This layer represents the external world and the variety of users the system serves.

- **Heterogeneous Clients:** KairosSNNI is designed to handle a mix of different devices, including resource-constrained IoT/mobile devices, tablets, laptops, and powerful desktop computers. Each device is characterized by its own hardware "Capability Vector."
- **Application Domains:** The framework is built to support high-stakes, privacy-sensitive tasks, such as:
    - **Healthcare:** Private medical diagnostics.
    - **Autonomous Driving:** Secure real-time perception.
    - **Finance:** Fraud detection and risk assessment.
    - **Smart Cities:** Private surveillance and urban management.
    - **Image Classification:** General-purpose secure vision services.

Summary of Operational Flow

Requests come in →**Admission Control** filters them →**Online Scheduler** sorts them →**Dispatcher** pins them →**Resource Manager** monitors them →**Profiler** learns from the request execution to update the **Execution Parameters**.

1. **Ingress:** Client submits the **Capability Vector (Cap_c)** and **Due Date ($D_{i,k}$)**.
2. **Admission:** The **Resource Manager** runs the **TTFR Handshake Probe** to verify the **Slowness Factor (θ_c)** before the request is enqueued.
3. **Negotiation:** The **State Machine** offers n_{alt}(Deep & Narrow) or applies the **Hostage Core Penalty (ϕ)**.
4. **SLO Filtering:** Implements the **Three-Tier Filter** (Tight, Static, and Dynamic **Virtual Finish Time**).
5. **Optimization:** The **Online Scheduler** uses **Simulated Annealing (SA)** or **DQN** to calculate the best permutation and **Malleable Core Sizing** to maximize Ψ.
6. **Proactivity:** The **Offline Scheduler** enforces **Strict Quotas** in the **1TB Project Space**.
7. **Execution:** The **Dispatcher** performs the **Triple Check** and **HoL Bypass**, then executes **Strict Physical Core Pinning** for the **Non-Preemptive** J_{on}.
8. **Feedback:** The **ReLU Audit Hook** captures ground-truth math latency and uses **EWMA** to update the Π **Profiling Table**.

Based on our finalized technical execution flow and the specific requirements for developer, here is the detailed and technically precise functional breakdown of the **KairosSNNI** components.

1. Applications Layer (The Request Source)

- **Multi-Tenant Gateway:** Represents diverse clients (IoT, Mobile, PC).
- **Developer Requirement:** The ingress API must strictly accept and parse the **Capability Vector (** Cap_c**)** and the request-specific **Due Date (** $D_{i,k}$**)** metadata alongside the encrypted payload ($I_{i,k}$).
- **Role:** It initiates the **Service Agreement** by providing hardware specs and timing targets.

2. Secure Inference Service Engine (The SISE Orchestrator)

The SISE is the core coding environment where the intelligent orchestration logic is implemented.

- **Admission Control:**
  - **The Gatekeeper:** Implements the **Multi-Tiered SLO Filter**.
  - **Logic:** It performs the **Practicality Check**, the **Static SLO-factor Filter**, and the **Dynamic VFT Filter** (Virtual Finish Time) to reject requests that are mathematically impossible or likely to cause system thrashing.
  - **Negotiation State Machine:** Triggers a model-shift proposal (n_{alt}) for weak clients before they enter the queue.
- **Profiler:**
  - **The Characterization Engine:** Maps task identifiers to specific execution demands using the Π **Profiling Table**.
  - **Learning Logic:** Implements the **EWMA (Exponential Weighted Moving Average)** feedback loop. It refines the **Client Slowness Factor (θ_c)** after every job based on ground-truth measurements from the execution backend.
- **Resource Manager:**
  - **The Hardware Sensor:** Acts as the high-precision sensor for the Profiler and the real-time guardrail for the Dispatcher.
  - **Functions:** Measures the **Time-to-First-Reply (TTFR)** during the initial handshake, captures **Energy (uJ)** via RAPL/Slurm, and provides instantaneous snapshots of physical core and memory availability.
- **Request Scheduler:**
  - **The Optimization Brain:** This module contains the **Simulated Annealing (SA)** and **Deep Q-Network (DQN)** algorithms.
  - **Operation:** It scans the **Global Request Queue** (arrival order) and transforms it into the **Final Sorted Queue** (optimized permutation). It determines both the execution order and the **malleable core count** ($1 \dots P_{\text{max}}$) for each job to maximize the utility metric Ψ
- **Dispatcher:**
  - **The Execution Interface:** Implements the **"Triple Check"** before dispatching a job. It verifies that physical cores are free, memory gaps are sufficient, and the specific J_{pre} file is present in storage.
  - **HoL Mitigation:** Implements **Non-blocking Head-of-Line Mitigation**. If any resource or cryptographic dependency is missing, it bypasses that job to keep the core system active.

3. Computational Infrastructure (The Resource Pool)

- **Model Catalog:** The persistent database containing the specifications for all supported neural networks, including their non-linear speedup profiles ($e_{\text{pre}}/e_{\text{on}}$).
- **1TB Project Space:** A high-speed, user-managed scratch area. It stores the **ephemeral, massive (GBs), and single-use** preprocessed files generated by the **Offline Scheduler**.

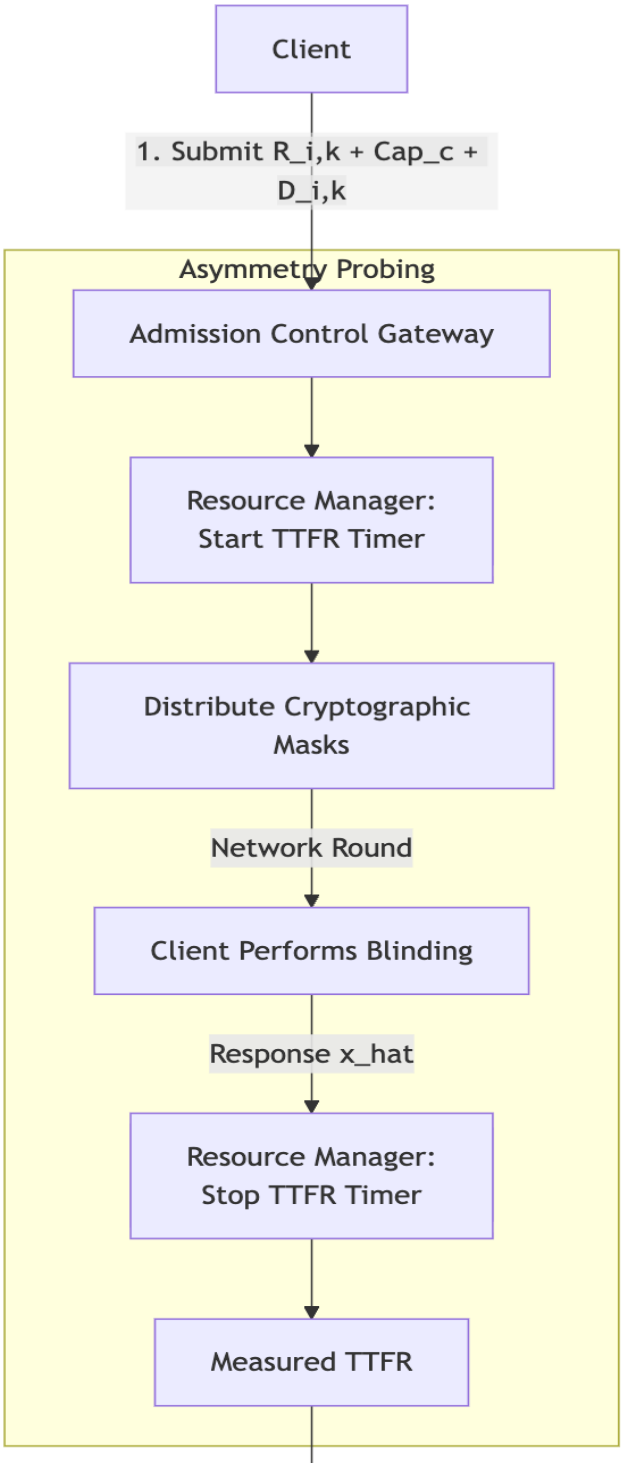
- **Strict Physical Core Pinning:** The hardware-level mechanism (e.g., taskset) used by the Dispatcher to bind SHARK processes to specific physical core IDs. This ensures **performance isolation** and prevents execution jitter.

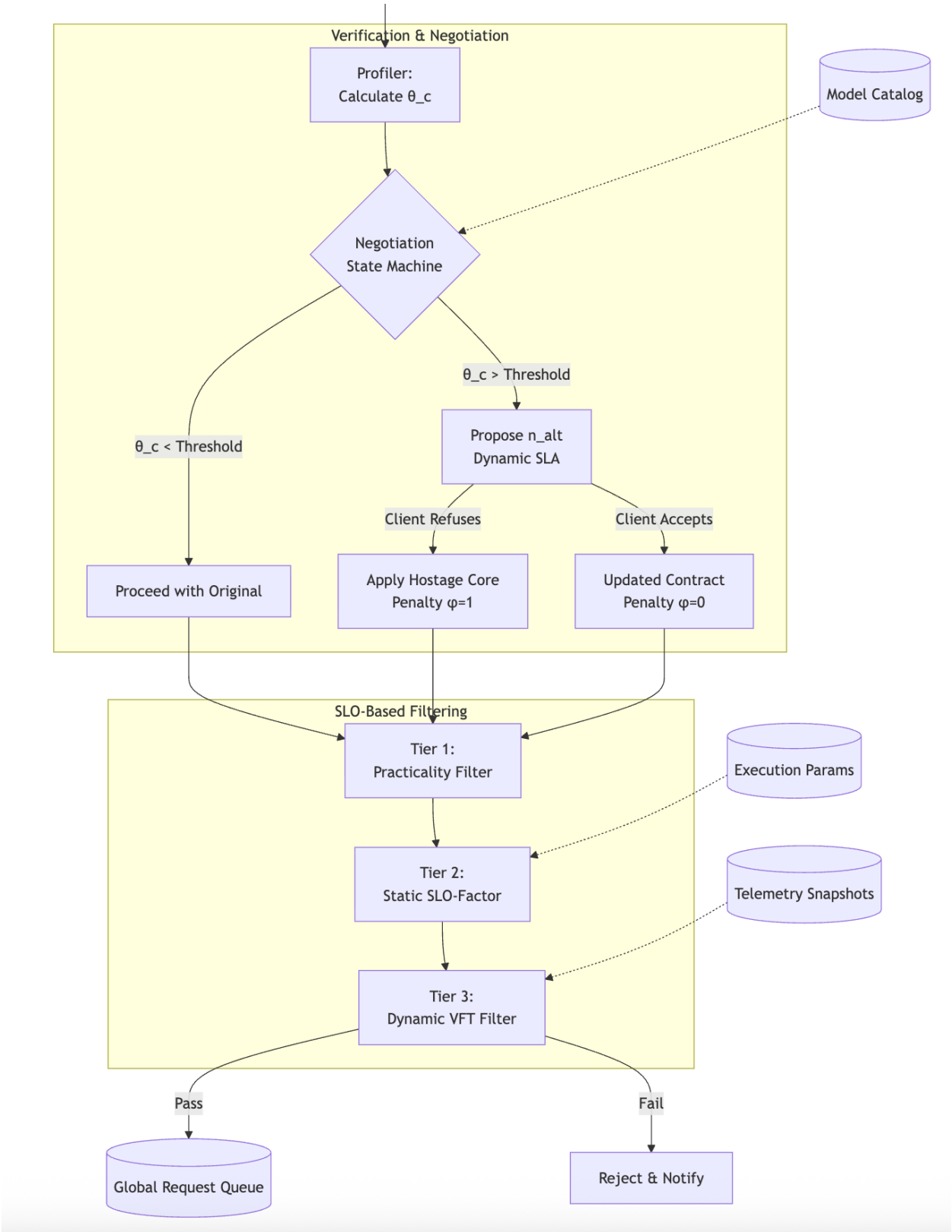
The Standard Operational Flow (Developer Reference)

1. **Ingress:** A **Client** submits an encrypted request ($R_{i,k}$) with Cap_{c} and $D_{i,k}$
2. **Validate: Admission Control** validates the request against the **Model Catalog** and performs **VFT filtering**.
3. **Profile:** The **Profiler** calculates the resource cost ($E_{i,c}, M_i$) based on the client's slowness factor.
4. **Optimize:** The **Request Scheduler** (SA/DQN) reorders the queue into an "opportune" sequence to maximize Ψ
5. **Triple Check:** The **Dispatcher** ensures the **Preprocessed File Manager** has the necessary file and the **Resource Manager** sees sufficient free cores/memory.
6. **Pin & Execute:** The **Dispatcher** pins the job to physical cores. The **SHARK Protocol** executes the inference non-preemptively on the **System Resources**.
7. **Learn:** The **Resource Manager** monitors the interactive rounds (ReLU). The **Profiler** learns from this ground-truth data to update the **Execution Parameters** for all future requests.

Flowcharts

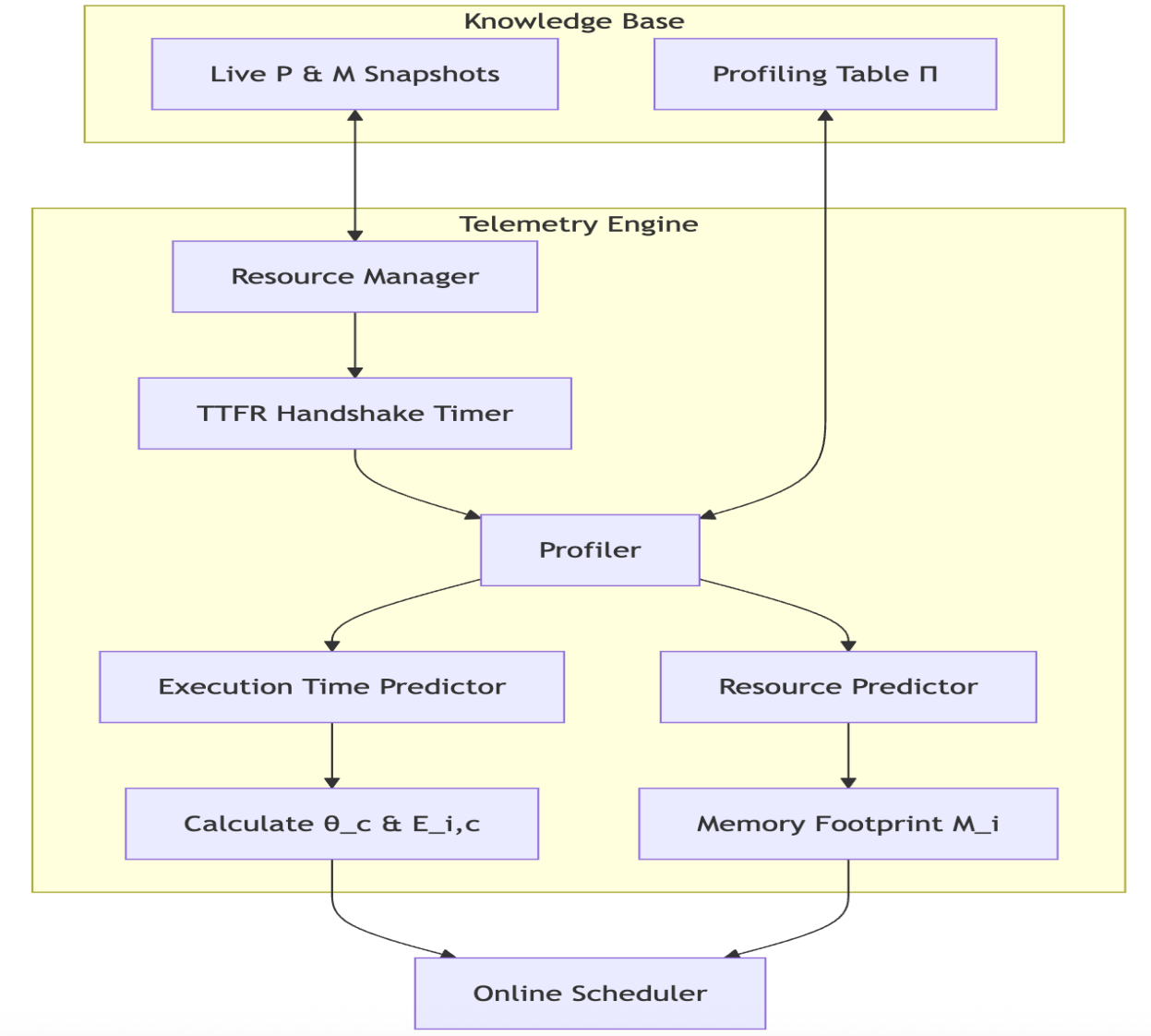
1. Ingress & Admission Control (The Gatekeeper with Handshake)




Verification & Negotiation
Profiler:
Calculate θ_c
Model Catalog
Negotiation
State Machine
θ_c > Threshold
θ_c < Threshold
Propose n_alt
Dynamic SLA
Client Refuses
Client Accepts
Proceed with Original
Apply Hostage Core
Penalty φ=1
Updated Contract
Penalty φ=0
SLO-Based Filtering
Tier 1:
Practicality Filter
Execution Params
Tier 2:
Static SLO-Factor
Telemetry Snapshots
Tier 3:
Dynamic VFT Filter
Pass
Fail
Global Request Queue
Reject & Notify

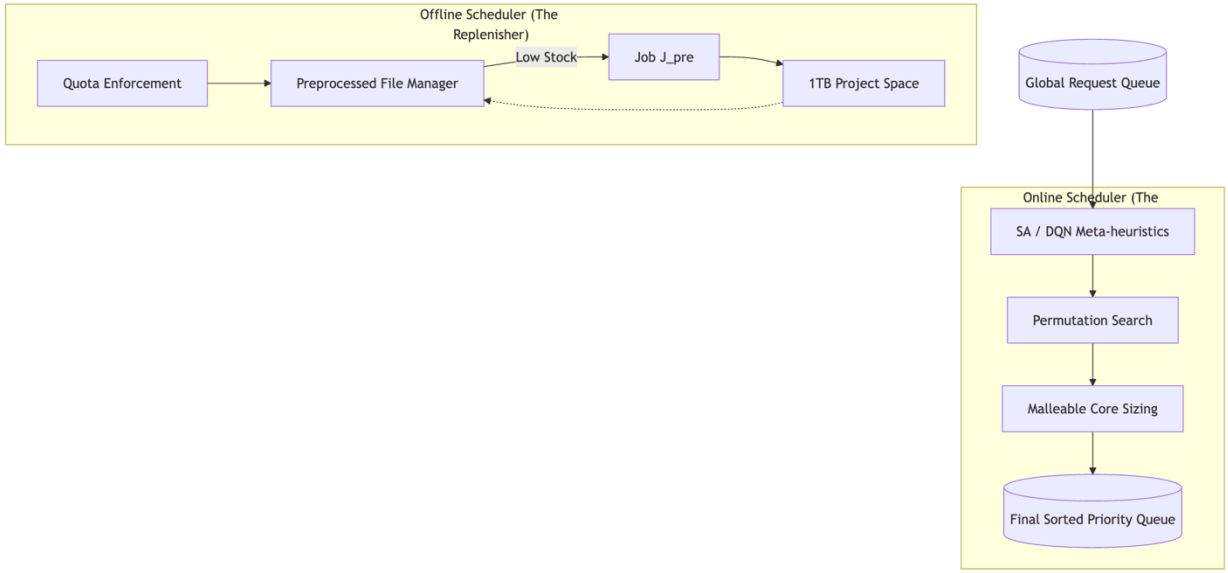
- **Handshake Timing:** The **Resource Manager** must wrap the mask distribution in a high-precision timer. This TTFR is the basis for the **Slowness Factor** θ_c
- **Active Verification:** The system does not just trust the client's **Capability Vector** (Cap_c). It uses the **TTFR** to verify it.
- **Negotiation Loop:** This is an asynchronous state. The developer must ensure the thread waits for the client's decision on the **Alternative Model** (n_{alt}).
- **Priority Penalty** (ϕ)**:** If negotiation fails, the request is flagged with $\phi = 1$. The **Online Scheduler** (the next component) will use this to lower its rank in the queue.
- **VFT Dependency:** The **Virtual Finish Time (VFT)** filter requires a snapshot of the current queue from the **Resource Manager** to make an admission decision.

2. **Telemetry Engine & Dynamic Profiler**



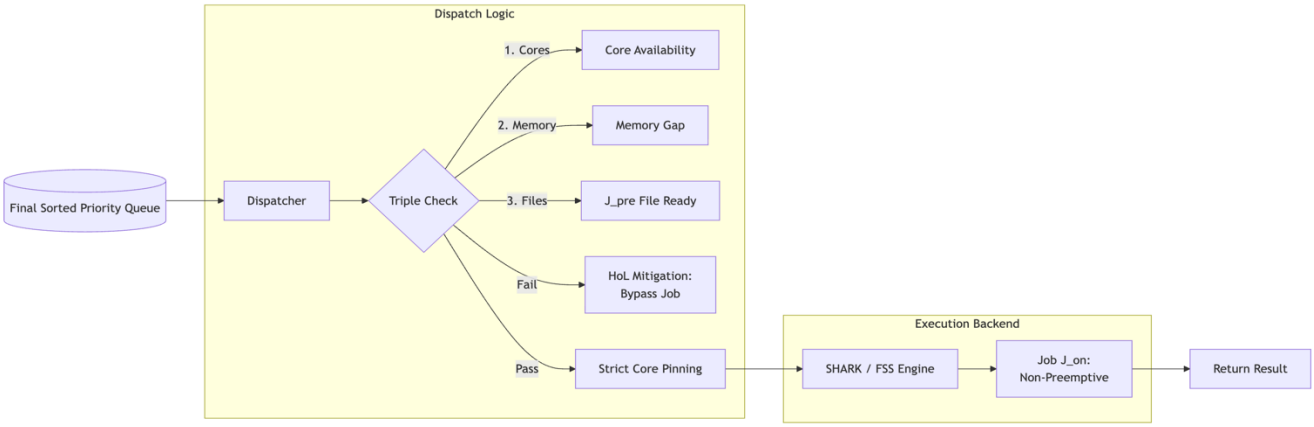
3. Request Scheduler: Proactivity & Optimization

This diagram distinguishes between the **Offline Scheduler/ Replenisher** (proactive) and the **Online Optimizer** (reactive).



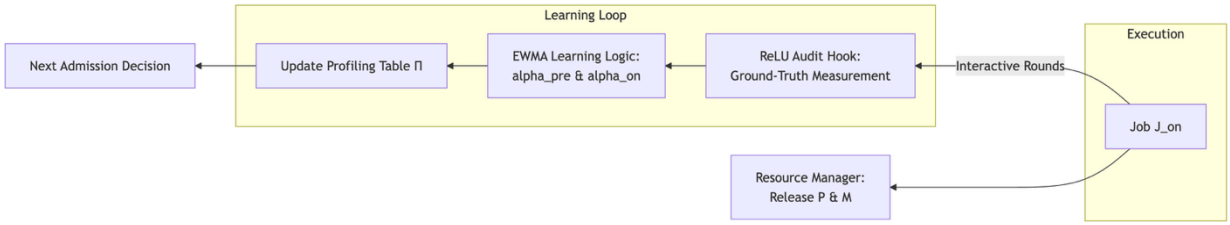
4. Dispatcher & Execution Backend

This component performs the **Triple Check** and enforces performance isolation.



5. Closed-Loop Feedback (The Learning Engine)

This component describes the **ReLU Audit Hook** and the **EWMA update** logic.



Part 1: Admission Control and Scheduling (Planning)

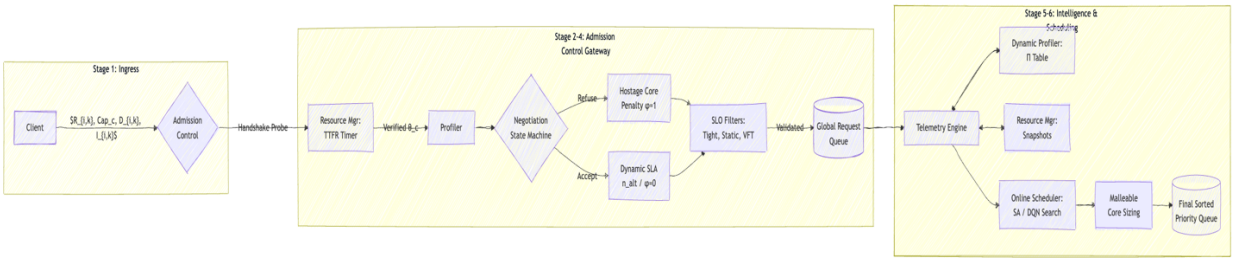


Diagram 1: The Brain (Planning)

- **Asymmetry Probing:** Uses **TTFR** to verify the **Capability Vector (Cap_c)**.
- **Negotiation:** Proposes n_{alt} or applies **Priority Penalty (ϕ)**.
- **Optimization:** Uses **SA/DQN** and **VFT** to reorder the queue for maximum Ψ
- **Malleability:** Determines the optimal core count ($1 \dots P_{max}$) during the search.

Part 2: Dispatch, Execution, and Learning (Action)
This diagram covers the flow from the sorted queue to physical execution and the closed-loop feedback.

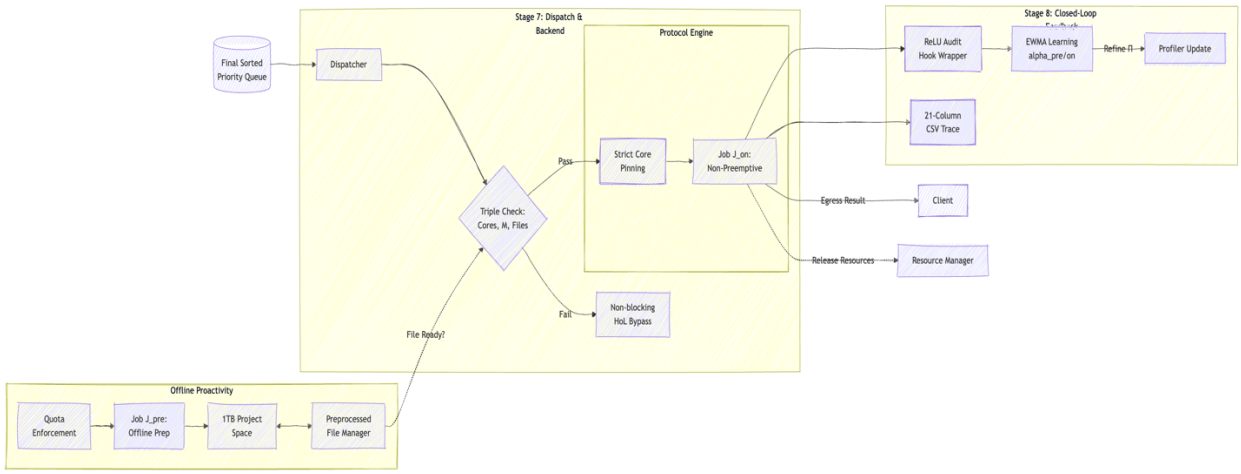


Diagram 2: The Muscle (Execution)

- **Proactivity:** The **Offline Scheduler** fills the **1TB Project Space** based on quotas.
- **Triple Check:** Ensures Cores, Memory, and offline Preprocessed Files from *J_pre* are ready in advance before pinning.
- **HoL Mitigation:** Bypasses jobs that aren't ready to keep the P cores busy.
- **Learning:** The **ReLU Audit Hook** captures ground-truth math latency to update the **EWMA** model.
- **Non-Preemption:** Ensures J_{on} runs to completion as an atomic unit.

Here is the technical justification of why your **KairosSNNI** "Handshake" does not violate the security or privacy of the SHARK protocol:

1. Data Privacy (Information-Theoretic Security)

SHARK uses **Additive Secret Sharing** for the input.

- **The Math:** To send input x, the client retrieves a random mask r (from the client.dat file provided by the dealer) and computes $\hat{x} = x + r$.
- **Why the Handshake is safe:** When the server measures **TTFR**, it sees the client sending $\hat{x}$. Because r is a uniform random value (a "One-Time Pad"), $\hat{x}$ looks like pure mathematical noise to the server. The server can time how long it took the client to compute $\hat{x}$, but the actual value of x remains hidden.

2. Model Privacy (Intellectual Property)

SHARK assumes a **Public Architecture** but **Private Weights (w_s)**.

- **Why the Handshake is safe:** During the handshake and negotiation phase, the server and client only discuss **Model IDs** (e.g., "AlexNet" or "M2"). The actual weights w_s are stored in the server.dat file and are never sent to the client. Intellectual property is never at risk during admission or negotiation.

3. Integrity and Correctness (MAC Verification)

You noted that SHARK uses **SPDZ-style authenticated secret sharing**.

- **The Process:** Every share has a corresponding **Message Authentication Code (MAC)**. At the end of the inference, the parties perform a "Commit-and-Open" check to verify these MACs.
- **Why the Handshake is safe:** The KairosSNNI handshake happens at the **Input Phase**. The MAC verification happens at the **Output/Egress Phase**. Measuring the speed of the input phase does not interfere with the mathematical ability to verify the result later.

4. The "Single-Use" Rule (Security Critical)

You identified that key files (server.dat/client.dat) are **single-use**.

- **The Requirement:** Each preprocessing output supports exactly one inference.
- **The Scheduler's Role:** This is where **KairosSNNI** is actually *better* for security. Your **Dispatcher** and **Resource Manager** enforce the single-use rule by deleting the file immediately after t_{fin} . Without a strict orchestrator, there is a risk that a system might accidentally reuse a file, which would leak the client's secret keys.

5. Why TTFR doesn't leak info (Timing Side-Channels)

A common concern is: "Does measuring the time reveal the data?"

- **In SNNI:** The operation

$$\hat{x} = x + r$$

is a **constant-time operation**. It takes the CPU the same number of cycles to add

$$x = 1$$

as it does to add

$$x = 1{,}000{,}000$$

Conclusion: Measuring the **TTFR** only reveals the **Client's Hardware Speed**, not the value of the input data.

The initial handshake in KairosSNNI is cryptographically compliant with FSS-based protocols like SHARK. It utilizes the standard **online-input phase**, where the client performs a local blinding operation ($x + r$) using pre-distributed masks. Since the server only observes the information-theoretically secure blinded value, input privacy is maintained. Furthermore, the orchestrator's role in managing single-use material directly supports the protocol's security model by ensuring that no preprocessing material is reused across concurrent requests.

Technical Summary for the Developer Contract:

- **Verify** that the Resource Manager deletes the specific line used in server.dat after every successful J_{on}.
- **Ensure** the Admission Control only sends the "Mask ID" or "Request for Input" signal, and never asks for the plaintext x.
- **Confirm** that the TTFR timer is based on the arrival of the blinded packet $\hat{x}$.

Problem formulation: **Technical Functional Specification**. **KairosSNNI** orchestrator.

1. Data Structures (Class Definitions)

The developer needs to implement two primary data structures to represent the hierarchy of work:

A. Task Profile Object (τ_i) — Static Configuration

- protocol_id ($\mathcal{P}_i$)
- model_id (n)
- batch_size (B_i)
- baseline_e_pre, baseline_e_on (from E_i)
- baseline_m_pre, baseline_m_on (from M_i)
- min_inter_arrival (t_i)
- alternative_model_id (n_{alt}) — *Used for the negotiation logic.*

B. Request Object ($R_{i,k}$) = ($c, \tau_i, t_{arr}, I_{i,k}, D_{i,k}, Cap_c, \phi_{i,k}$) -) — Dynamic Arrival

- **Client_id (c):** The specific client ID issuing the request.
- **Task Reference (τ_i):** A pointer to the static task blueprint (Protocol, Model, etc.).
- **Release Date (t_{arr}):** The absolute arrival timestamp at the server gateway.
- **Payload ($I_{i,k}$):** The batch of encrypted inputs, where the count $| I_{i,k} | = B_i$
- **Due Date ($D_{i,k}$):** The request-specific relative deadline (SLO) for batch completion.
- **Capability Vector (Cap_c):** The hardware self-report provided by the client. — *Includes device_class, cpu_freq, ram, net_type.*
- **Priority Penalty ($\phi_{i,k}$):** A binary flag (0 or 1) assigned by Admission Control if the client refuses a recommended model-shift during negotiation.
- slowness_factor: (θ_c) — *Calculated by Profiler using* Cap_c

1.2. **Capability Vector Characterization (Cap_c)**

the client provides a structured hardware profile. Cap_c **:= (class, cpu, ram, net)**

- **class (Device Category):** An integer ∈ $\{0,1,2,3,4\}$ representing the hardware class:
    - 0: IoT
    - 1: Mobile
    - 2: Tablet
    - 3: Laptop
    - 4: Desktop
- **cpu (Processing Speed):** A float representing the client's CPU clock frequency in GHz.
- **ram (Memory Capacity):** An integer representing the available client-side memory in MB.
- **net (Network Type):** An integer ∈ $\{0,1,2,3\}$ representing the connection class:
    - 0: LAN
    - 1: WiFi
    - 2: 5G
    - 3: WAN

2. The Execution Engine Logic

The developer must implement the "Physics" of the SNNI protocol:
A. The Asymmetric Burst Time Formula
The scheduler must use this to predict how long a job will hold a core: $E_{i,c} = e_{pre} + (\theta_c \cdot e_{on})$

B. Decision Variables
For every SNNI phase (J_{pre} and J_{on}), the developer must track:

- start_time: The moment the Dispatcher pins the job.
- finish_time: Calculated as start_time + duration.
- **Constraint:** finish_time_pre <= start_time_on.

3. Operational Constraints (The "Guardrails")

The developer needs to implement the following checks in the ResourceManager and Dispatcher modules:

- **Non-Preemption:** Once a process is launched via exec, do not allow the scheduler to pause or migrate it.
- **Malleable Core Assignment:** Online jobs can use a variable number of cores $p_J \in [1, P_{max}]$.
- **System Capacity:**
    - Sum(p_J) <= P (Physical core limit).
    - Sum(m_phase) <= M (Physical RAM limit, including current active jobs and the proactive buffer).

4. Performance Metrics (The Telemetry Logger)

The developer must create a logging function that captures the following for every single input j:

1. **Completion Time (CT):** Timestamp when J_{on} exits.
2. **Turnaround Time** TAT: $CT - t_{arr}$
3. **Waiting Time (w):** $TAT - E_{i,c}$
4. **Tardiness:** $max(0, TAT - D_{i,k})$
5. **Success Flag (χ):** 1 if $TAT \leq D_{i,k}$, else 0.

5. The Optimization Goal (Ψ)

The **Online Scheduler (SA/DQN)** must use the following utility function to reorder the priority queue:

$$\text{Maximize } \Psi = \frac{\sum No.\,of\ SLOs \text{ Successes}}{\sum \text{Latency /TAT Summation}}$$

- **Developer Tip:** High Ψ means the system is completing many jobs on time while keeping individual wait times low. Low Ψ suggests Head-of-Line (HoL) blocking or resource waste.
- I would like to extend our objectives to multi objective task scheduling for energy efficient and cost effective (if possible)

6. Integration Checklist for the Developer

Provide this list as "To-Do" items:

Metadata Parsing: Ensure the TCP gateway can parse Cap_c and $D_{i,k}$
Telemetry Probe: Implement the TTFR timer during the initial mask distribution.
Negotiation Loop: Implement the if (theta > threshold) model-shift proposal.
Triple Check: In the Dispatcher, verify **Cores + Memory + Preprocessed File** exist before starting.
HoL Mitigation: If a job fails the Triple Check, **bypass** it and move to the next job in the priority queue.

Summary of Scientific Labels for the Developer

Explain that they are building a scheduler.

- We know the costs (E, M) at arrival.
- **Online-time:** Jobs arrive sporadically.
- **Stochastic:** The duration is a random variable (X) influenced by the client (θ_c).

Algorithmic Suite

"We have categorized and refined six scheduling algorithms to evaluate the 'cost of SLO-awareness':

- **Baselines:** FCFS and SJF-Aging.
- **Urgency-Based:** EDF and Least Slack Time (LST).
- **Intelligent Optimizers: Simulated Annealing (SA)** with 3D perturbation (Swapping, Sizing, and Dwell-shifting) and **DQN (Reinforcement Learning)** using a Pointwise Ranking policy."

5. The Feedback & Learning Loop

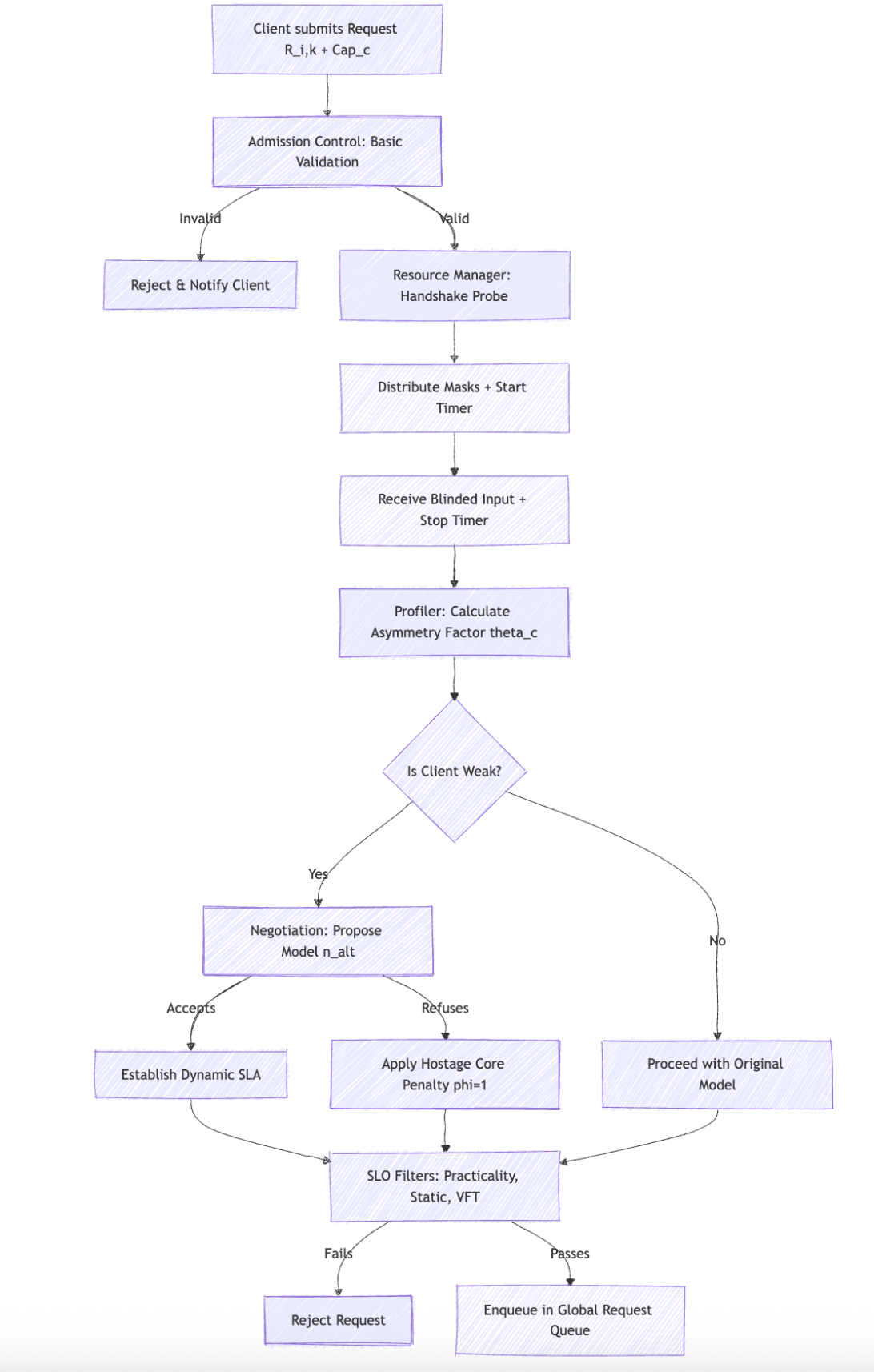
"To ensure the system adapts to real-world network jitter:

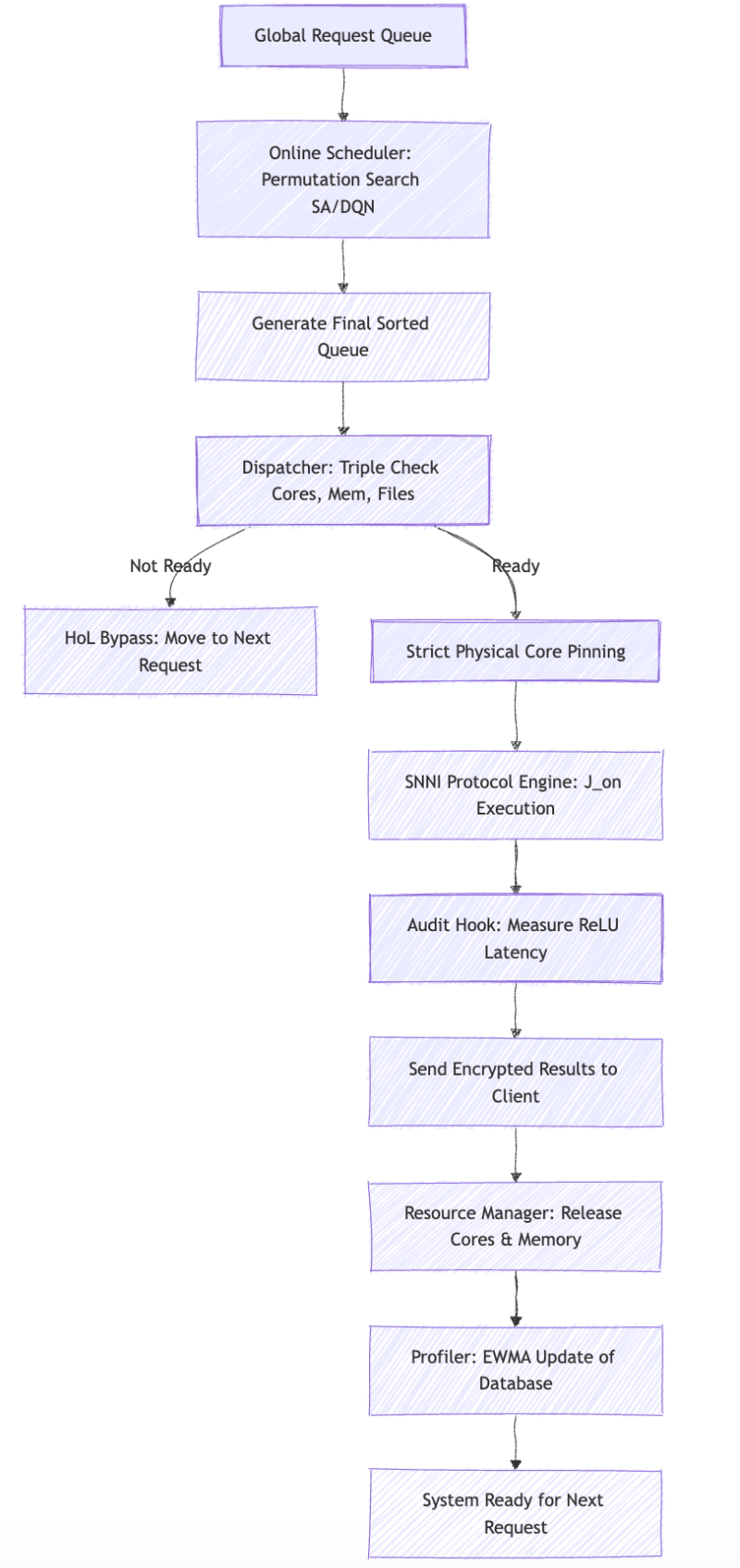
- **Initial Probe:** We use the protocol's own handshake (TTFR) to verify client capacity before enqueuing.
- **ReLU Audit:** We use an external execution wrapper to measure ground-truth compute speed during online inference J_on execution.
- **Learning:** These measurements update our Profiling Table via an **Exponential Weighted Moving Average (EWMA)** so the system gets smarter with every request."

Workflow in detail

Diagram 1: Ingress, Admission Control, and Negotiation

This diagram focuses on the "Gatekeeper" logic: identifying client slowness and establishing the service agreement.


Client submits Request R_i,k + Cap_c
Admission Control: Basic Validation
Invalid
Valid
Reject & Notify Client
Resource Manager: Handshake Probe
Distribute Masks + Start Timer
Receive Blinded Input + Stop Timer
Profiler: Calculate Asymmetry Factor theta_c
Is Client Weak?
Yes
No
Negotiation: Propose Model n_alt
Accepts
Refuses
Establish Dynamic SLA
Apply Hostage Core Penalty phi=1
Proceed with Original Model
SLO Filters: Practicality, Static, VFT
Fails
Passes
Reject Request
Enqueue in Global Request Queue


Global Request Queue
Online Scheduler: Permutation Search SA/DQN
Generate Final Sorted Queue
Dispatcher: Triple Check Cores, Mem, Files
Not Ready
Ready
HoL Bypass: Move to Next Request
Strict Physical Core Pinning
SNNI Protocol Engine: J_on Execution
Audit Hook: Measure ReLU Latency
Send Encrypted Results to Client
Resource Manager: Release Cores & Memory
Profiler: EWMA Update of Database
System Ready for Next Request

Appendix:

for Developer Implementation:

1. **Asymmetric Resource Handling:** The Profiler must use the TTFR Timer from the Resource Manager to calculate θ_c and $E_{i,c}$ before the Online Scheduler makes decisions.
2. **Malleable Allocation:** The Online Scheduler determines core count, and the Dispatcher enforces it via Strict Physical Core Pinning.
3. **Non-Blocking HoL Mitigation:** The Dispatcher must implement the logic to skip a job in the Priority Queue if the Preprocessed File Manager reports the J_{pre}file is missing from the 1TB Project Space.
4. **Ephemeral Memory Management:** The Triple Check ensures that the sum of m_phase for all active jobs plus the PFM buffer does not exceed memory M.
5. **Stochastic Learning Loop:** The ReLU Audit Hook must be an external wrapper that feeds real-time latency back to the Profiler to update the EWMA smoothing of slowness factors.
6. **Non-Preemptive Execution:** Once J_on is started by the Protocol Engine, the Resource Manager must flag those cores as "locked" until the exit code is received.
7. **Negotiation State Machine:** Built into Admission Control. It uses the Model Catalog to propose an n_alt (e.g., M2 for AlexNet) to "Weak" clients before enqueuing.
8. **Priority Penalty (ϕ):** If negotiation is refused, the Admission Control tags the request with $\phi = 1$, which the SA/DQN Optimizer uses to lower the job's priority.

#	Item	Question	Context / Impact	Client Answers
C-01	ReLU Audit Hook Scope	Is the ReLU Audit Hook required for the initial delivery? It requires modifying SHARK source code (relu.cpp / relutruncate.cpp) which may affect cryptographic protocol correctness.	HIGH RISK: Modifying SHARK crypto code may break batch_check MAC verification. Risk R1 in the Implementation Plan rates this as High Impact. Recommend deferring to Phase 5.	**Use External Wrapper.** Do not modify internal relu.cpp. Capture timestamps immediately before and after the function call in the SISE layer. You do not need to modify the internal cryptographic kernels (relu.cpp), instead change the requirement to a Function Wrapper, so the security proof of SHARK remains valid.

				We will simply 'stopwatch' the function call from the outside. This gives us the θ_c update data we need without any risk of breaking the MAC verification. We can not eliminate this because Without it, our DQN Agent has no "Reward" signal based on actual compute performance.
C-02	Capability Vector Protocol	Should Cap_c be transmitted in the existing TCP handshake format ("r_model_batch_slo"), or does the Client want a new JSON/protobuf handshake protocol?	Current format: "r_model_batch_slo" (4 underscore-delimited fields). Adding device_class/cpu_freq/ram/ network changes the wire protocol. See KairosServer.cpp:90-96 for current parsing.	**Extend Delimited Protocol.** Use 8-field string: r_model_batch_slo_class_cpu_ram_net. Ensure microsecond-scale parsing in KairosServer.cpp.
C-03	θ_c Threshold Value	What slowness factor threshold should trigger the negotiation state machine?	Contract says "If θ_c exceeds threshold" but does not specify the value. Suggestion: θ_c > 1.5 (client 50%+ slower than baseline). Also: should TTFR be measured per-session or per-request?	**Threshold** $\theta\_c \geq 1.5$ triggers negotiation. Measure TTFR **per-request** using ResourceManager timers to capture transient edge jitter.
C-04	n_alt Model Mapping	Which alternative models map to which primary models? e.g., AlexNet → ? Should this be in profile.cfg?	Contract mentions n_alt in Task Profile but no mapping table. Current profile.cfg has 4 models × multiple batches (simc1, hinet, simc2, alexnet). Need explicit mapping.	Store this mapping in the n_alt column of profile.cfg if n = AlexNet then n_alt = SimC2 (M2) or

				HiNet. for Clients with low RAM. if n = VGG16 then n_alt = HiNet or ResNet50 or SimC2 (M2). for Clients with slow CPUs/High Network Jitter. if n = D4 or Standard ResNet50 then n_alt = SimC1 (M1) or SimC2 (M2). for Clients on mobile/IoT classes. AlexNet to HiNet or DeepSecure or SimC2 (M2) Rule: For every "Heavy" model n, define a "Light" alternative n_alt Logic: Check: θ_c >Threshold. Lookup: Find τ_i → n_alt. Action: Propose n_alt with a revised D (i,k)(Due Date).
C-05	EWMA α Value & Configurability	Should EWMA_ALPHA be configurable in config.cfg? What default? Contract specifies range 0.1–0.8.	Currently hardcoded as 0.2 in KairosServer.cpp:179 and :235. Higher α = more responsive but volatile. Also: should α differ for inference vs preprocessing profiling?	**Differentiated Learning Rates:** $\alpha_{pre} = 0.2$ (Server stability), $\alpha_{on} = 0.4$ (Client responsiveness). Store in config.cfg.
C-06	Hostage Core Duration	Should the dispatcher actively extend timeout for slow clients (J_on	If θ_c × e_on is used in VFT estimation, it affects admission control. Currently,	**Active Lease Enforcement.** The Dispatcher must use $(\theta_c \cdot e_{on})$

		= θ_c × e_on), or is this informational only for logging?	timeout is fixed at 600s (KairosServer.cpp:171). If θ_c modifies this, resource allocation changes significantly.	as the dynamic execution lease. Re-calculate VFT based on this extended duration.
C-07	Practicality Filter vs VFT	The VFT Filter (Tier 3) subsumes the Practicality Filter (Tier 1) since VFT ≥ e_on always. Should Tier 1 still send a distinct rejection message?	Currently only VFT filter is implemented (KairosServer.cpp:103-126). Adding distinct error messages per tier requires additional code paths. Confirm message format.	**Sequential Exit.** Perform Tier 1 (Practicality) check before Tier 3 (VFT). Use distinct error codes (403 for Practicality, 404 for Congestion).
C-08	Online DQN Fine-Tuning	Is online fine-tuning of the DQN model in C++ required? Contract says "pre-trained or online RL agent".	Current: offline Python training → binary export → C++ inference. Online fine-tuning requires implementing backpropagation, replay buffer (50K+ entries per Paper 1), and target network in C++. Significant engineering effort.	**Hybrid Asynchronous Learning.** No C++ backprop. Log (s, a, r, s') tuples; use Python background process to update weights; implement C++ Hot-Reload.
C-09	Temporal Memory Logging	Contract §9 requires logging total RAM every 100ms. Continuous background thread or per-request sampling?	Continuous 100ms sampling generates ~36K entries/hour. Currently: per-request snapshot in Logger::logJob() at completion time. SystemMonitor reads /proc/meminfo. Confirm if continuous thread is required.	**Continuous Background Thread.** Poll /proc/meminfo every 100ms. Log mem_active vs mem_buffer to visualize the Spatio-Temporal memory wall.
C-10	CSV Log Format Extension	Should logger output 19+ columns (Contract format) or maintain current 14-column format?	Current columns (Logger.cpp): Arrival_TS;Start_TS;Finish_TS;Model_Batch;Wait_Time;Cores_Assigned;CPU_L	**Expanded 21-Column Trace.** Append RequestID, phase-specific start/finish timestamps, θ_{est}, θ_{act}

			oad;Mem_GB;Energy_uJ;Requested_SLO;Actual_TAT;SLO_Diff;SLO_Met;Exit_Code. Contract adds: request_id, t_start_pre, t_fin_pre, theta_est, theta_act, penalty. Recommend appending new columns.	, and Priority Penalty ϕ to existing schema.
C-11	DQN Normalization Constants	DQN features normalize by hardcoded values (24.0 for cores). Should these be configurable for different hardware?	DQNScheduler.hpp:167-170 uses 24.0 for CPU normalization. On different hardware (e.g., 96-core server), these constants need updating. Should we derive from config TOTAL_CORES?	**Configuration-Driven.** Derive state normalization denominators from TOTAL_CORES and TOTAL_MEMORY_MB in config.cfg instead of hardcoded 24.0.
C-12	SA Perturbation Strategy	Should additional perturbation strategies from the reference paper be implemented for SA?	Paper defines 3 types: swap_random_pair (implemented), squeezeLastIter, delayNextIter. The latter two operate on LLM iteration-level scheduling. If SNNI-adapted versions are desired, specify the adaptation strategy.	**Three-Dimensional Search.** Implement swap_jobs, adjust_malleable_cores, and shift_dwell_time to manage memory-compute coupling.
C-13	Ψ Metric Computation Point	Where should the Ψ = Successful_SLOs / Aggregate_TAT metric be computed — in the server (real-time) or as a post-hoc analysis script?	Real-time computation requires an additional background thread. Post-hoc analysis uses the CSV log. Both are feasible. Recommendation: post-hoc script + optional real-time endpoint.	**Dual-Tier.** Real-time estimation in OnlineScheduler for decision making; raw timestamp logging for post-hoc Python analysis scripts.

C-14	VFT Safety Margin Range	What VFT_SAFETY_MARGIN values should be benchmarked? Current default: 1.2 (config.cfg).	A margin of 1.0 = no safety buffer (aggressive admission). 1.2 = 20% slack. Contract does not specify a value. Recommend benchmarking [1.0, 1.1, 1.2, 1.5] with Poisson load.	**Benchmark Range.** Support a configurable float σ . Test and report values $[1.0, 1.1, 1.2, 1.3, 1.5]$ in the evaluation section.
C-15	DQN Fallback to LST	If DQN weight file is missing or invalid, DQNScheduler automatically falls back to LST (Least Slack Time) heuristic. Contract §5.3 does not specify fallback behavior for DQN.	INDEPENDENT DESIGN DECISION. DQNScheduler.hpp:162-164 returns -slack_sec when weightsLoaded==false. This ensures the system never stops scheduling even without a trained model. Client should confirm this is acceptable vs. rejecting all requests when DQN weights are unavailable.	**LST Default.** If weights are invalid, default to Least Slack Time. Log active mode as Scheduler_Mode=1 (FALLBACK) in the trace.
C-16	SA Multi-Objective Cost Function	Contract §5.3 says SA "minimizes total number of missed deadlines." Our implementation uses a 3-component cost function: (1) tardiness² penalty, (2) memory pressure, (3) slack urgency. This goes beyond the Contract specification.	INDEPENDENT DESIGN DECISION. SAScheduler.hpp:92-143. The paper (Huang et al.) uses G = n/t (SLO count / latency). We extended to multi-objective. Memory pressure component (90% threshold, 10× overflow penalty) and slack urgency bonus are not in Contract or reference paper. Confirm: keep multi-	**Multi-Objective Energy.** Minimize $Cost = Tardiness^2 + (10 \times MemPenalty) + SlackBonus$. Weighted memory penalty triggers at 90% utilization.

			objective, or simplify to paper's G=n/t?	
C-17	SA Lazy Re-optimization	SA only re-runs the optimization when new jobs arrive (needsOptimization flag). If no new jobs arrive, the previous ordering is preserved without re-running SA.	INDEPENDENT DESIGN DECISION. SAScheduler.hpp:20-24 (push sets flag), :29 (checked in sortQueue). Contract does not specify when SA should re-run. Alternative: re-optimize on every popReadyJob call (more CPU overhead but adapts to changing system state). Confirm preferred behavior.	**Hybrid Event-Triggered.** Re-run SA on Request Arrival AND Job Completion (resource release). Enforce a 50ms minimum re-optimization interval.
C-18	SA Windowed Optimization	SA only optimizes the first SA_WINDOW_SIZE (default: 20) jobs in the queue, not the entire queue. Contract §5.3 says "re-orders the queue" (implies full queue).	INDEPENDENT DESIGN DECISION. SAScheduler.hpp:32. Full queue optimization is $O(n^2)$. Windowed approach is $O(w^2)$ where w=20, keeping overhead < 1ms. The reference paper (Huang et al.) processes the entire batch. Confirm: windowed (current) or full-queue (paper-aligned)?	**Full-Queue Visibility.** Optimize the entire queue up to a safety depth of 100 jobs. Abandon windowed approach to ensure global Ψ maximization.
C-19	DQN Per-Job Priority Scoring	Contract §5.3 says DQN should "select the next job based on queue state and hardware load." Our DQN assigns a priority SCORE to	INDEPENDENT DESIGN DECISION. DQNScheduler.hpp:54-60. Reference papers use DQN for action-selection	**Pointwise Ranking.** Use a 5-input, 1-output MLP with 64-32-1 hidden layers. Sort the queue by the scalar priority score output by the network.

		each individual job and sorts by score. This is a regression approach, not a Q-value action-selection.	(choose which job to dispatch). Our approach trains a value estimator that predicts reward per job. The architecture (5→16→8→1) is smaller than papers (2×256). Confirm: per-job scoring is acceptable, or should we implement full DQN action-selection over the queue?	
C-20	DQN Binary Weight Export Format	We defined a custom binary format for DQN weights: [4B magic "DQN1"][4B×4 dims][964B float32 weights] Contract does not specify the weight transport format.	INDEPENDENT DESIGN DECISION. DQNScheduler.hpp:38, train_dqn.py:122-128. Alternative formats: ONNX Runtime (heavier dependency), PyTorch JIT (requires libtorch in C++), plain text CSV. Current binary format is minimal (984 bytes, zero dependencies). Confirm: acceptable, or prefer standard format (ONNX)?	**DQN2 Header Specification.** Include 16-byte header with Magic Number (DQN2), Architecture Hash, and a CRC32 checksum for payload integrity.
C-21	Elastic Core Allocation	Contract §3.3 says "Malleable Allocation: determine optimal core assignment (1…P_max)." Our implementation uses acquireCoresElastic() which allocates fewer cores if the requested amount is unavailable,	INDEPENDENT DESIGN DECISION. ResourceManager.hpp:24 (acquireCoresElastic). Contract says "determine the optimal core assignment" but does not specify behavior when fewer cores are available.	**Scheduler-Strict.** Do not perform "Elastic Shrinking." Jobs must receive the exact core count p_J assigned by the scheduler; otherwise, Bypass.

		rather than waiting.	Current: allocate whatever is free (elastic). Alternative: wait for full allocation (strict). Confirm preferred behavior.	
C-22	600-Second Execution Timeout	Every inference process has a hardcoded 600-second (10 min) timeout cap. Contract §4.2 says "non-preemption" but does not specify a timeout.	INDEPENDENT DESIGN DECISION. KairosServer.cpp:171 (ProcessLauncher::runShellCommand(cmd, 600)). This is a safety mechanism to prevent zombie processes. Contract says inference runs to completion, but does not address the case where a process hangs. Confirm: 600s acceptable? Should it be configurable? Should θ_c × e_on affect this?	**Dynamic Watchdog.** Replace 600s with $T_{lease} = (\theta_c \cdot e_{on}) \cdot 1.2$. Apply a global system absolute cap of 1800s.
C-23	95% RAM Dispatcher Cap	The dispatcher limits inference dispatch when total system memory exceeds 95% of total RAM. Contract §4.3 says "Total Memory ≤ M" but does not specify the safety margin.	INDEPENDENT DESIGN DECISION. KairosServer.cpp:133 (ram_limit = snap.total_mem_gb * 0.95). This 5% headroom prevents OOM from OS overhead. Confirm: 95% acceptable? Should this be configurable (e.g., MEM_SAFETY_FACTOR in config.cfg)?	**Configurable Headroom.** Use MEM_SAFETY_THRESHOLD=0.90 and enforce a 2GB hard reserve for OS/Buffer operations. Bypass jobs if RAM check fails.

C-24	VFT Rejection with SUGGESTED_SLO	When VFT admission rejects a request, the server returns REJECTED_VFT:SUGGESTED_ SLO:<value> to the client. Contract §3.1 says "reject" but does not specify the rejection message format or SLO suggestion.	INDEPENDENT DESIGN DECISION. KairosServer.cpp:122. This is a proto-negotiation: the server tells the client what SLO would be accepted. Contract §3.2 describes a full Negotiation State Machine with n_alt proposals, but SUGGESTED_SLO is a simpler mechanism. Confirm: is this sufficient, or must the full §3.2 negotiation protocol be implemented alongside?	**Full §3.2 Implementation.** Rejection must return SUGGESTED_MODEL (n_{alt}) and SUGGESTED_SLO (VFT). Handle 500ms client timeout.
C-25	Dynamic Profile Auto-Save	EWMA-updated execution times are automatically saved to logs/dynamic_profile.cfg after every job completion. Contract mentions EWMA learning but not persistence.	INDEPENDENT DESIGN DECISION. KairosServer.cpp:198 (saveDynamicProfile called per job). This enables warm-start: if the server restarts, it can reload learned profiles. But it also causes disk I/O per job. Confirm: auto-save acceptable? Should frequency be configurable (e.g., every N jobs or every T seconds)?	**Asynchronous Checkpointing.** Save learned profiles every 10 jobs or 30 seconds using an atomic write-and-rename strategy to avoid disk I/O jitter.
C-26	EWMA Update Only on Success (exit_code==0)	Dynamic profile is only updated when a job exits successfully. Contract §3.2 says "EWMA Learning" but does not specify behavior	INDEPENDENT DESIGN DECISION. KairosServer.cpp:175 (if rc == 0). Failed jobs may have anomalous	**Differentiated Updates.** Update Profiler on Success (rc=0) and Timeout (rc=penalty). Ignore binary crashes/OS errors to prevent database pollution.

		on failed jobs.	durations (e.g., immediate crash = 0ms, timeout = 600s). Including them would corrupt the EWMA estimate. Confirm: success-only learning is acceptable.	
C-27	CustomScheduler (CUSTOM mode)	A 7th scheduler mode "CUSTOM" exists as an FCFS placeholder. Contract §5 lists 6 algorithms (FCFS, SPT, EDF, LST, SA, DQN) and says "Add a few more related techniques if possible."	INDEPENDENT DESIGN DECISION. CustomScheduler.hpp — empty sortQueue (FCFS behavior). This satisfies the "plug-and-play interface" requirement but has no custom logic. Confirm: should additional algorithms be implemented here? Candidates: Weighted Fair Queuing, Multi-Level Feedback Queue, or Genetic Algorithm.	**Implement SPLS.** The "CUSTOM" mode shall use the State-Prioritized Least-Slack heuristic to prioritize buffer-ready jobs over urgency.
C-28	Port Allocation: Round-Robin Entropy	Ports are allocated using round-robin cycling through a pre-allocated range (BASE_PORT to BASE_PORT+PORT_RANGE). Contract §3.4 mentions "ports" but not the allocation strategy.	INDEPENDENT DESIGN DECISION. ResourceManager.hpp:59-61. Round-robin prevents port reuse collisions from TIME_WAIT states. Alternative: random selection, sequential, or OS-assigned ephemeral. Confirm: round-robin is acceptable.	**Round-Robin Entropy.** Increment an atomic port counter through the range 5000-6000 to prevent TCP TIME_WAIT collisions during bursts.

C-29	Hybrid Energy Measurement (RAPL + Slurm)	Energy is measured via hardware RAPL registers first; if access is denied, falls back to Slurm sacct accounting. Contract does not mention energy measurement at all.	INDEPENDENT DESIGN DECISION. SystemMonitor.hpp: 31-37. Energy is logged in the CSV (Energy_uJ column) but Contract §9 does not list energy as a required column. Reference DQN papers use energy in the reward function (R = w_1·f_energy + ...). Confirm: keep energy logging? Useful for future DQN reward enhancement per reference papers.	**Hybrid RAPL+Slurm.** Log energy in Microjoules (uJ) as the delta between start and end snapshots. Include as a mandatory CSV column.
C-30	Detached Worker Threads	Worker threads for inference are detached (.detach()) rather than joined. This means the main process cannot wait for worker completion. Contract §4.2 says "non-preemption" but does not specify the threading model.	INDEPENDENT DESIGN DECISION. KairosServer.cpp:200. Detached threads simplify the dispatcher loop (no join bookkeeping) but mean unclean shutdown if the server is killed during active inference. Alternative: thread pool with join-on-shutdown. Confirm: acceptable for the research artifact scope?	**Detached Workers with Callbacks.** Use .detach() but enforce resource release and logging via a mandatory completion callback in the lambda.
C-31	Template Method Pattern for Schedulers	All schedulers share IScheduler::popReadyJob() which calls virtual sortQueue() + file-aware bypass logic. Contract §3.4 says	INDEPENDENT DESIGN DECISION. IScheduler.hpp:24-50. The bypass loop (lines 37-48) iterates the sorted queue and skips jobs	**Centralized Triple Check.** The base IScheduler::popReadyJob() must verify File Readiness, Core Gaps, and Memory Gaps before popping.

		"HoL Mitigation: non-blocking bypass logic" but does not specify the design pattern.	without ready files. This is our HoL mitigation implementation. All 7 schedulers inherit this logic. Confirm: this satisfies Contract §3.4 HoL Mitigation requirement.	
C-32	Profiler: EWMA Only (No Standard Deviation)	Contract Profiling §5 says "Moving Average + Standard Deviation." Current implementation tracks only EWMA (moving average). Standard deviation is NOT computed or stored.	GAP vs CONTRACT. KairosServer.cpp:179-180 — only stores EWMA (alpha=0.2). Types.hpp ModelProfile has dynamic_inf_ms/dynamic_p re_ms (both atomic<long>) but no stddev fields. Adding stddev requires an additional atomic field and Welford's online algorithm. Confirm: is stddev required for initial delivery?	**Implement Welford's Algorithm.** The Profiler must track and store the **Standard Deviation (σ)** for both phases to enable stochastic analysis.

Regarding **Q-1:** Cap_c

Implementation, follow these concise instructions:

- **Timeline:** Implementation is **Mandatory for Phase 1**.
- **Reason:** The scientific novelty of the paper (asymmetry awareness) cannot be proven without this data.
- **Parser Logic:** Implement a **dual-format parser** in KairosServer.cpp.
  - **8-Field String:** Parse all fields into the Request object.
  - **4-Field String (Legacy):** Do not reject. Instead, apply **Symmetric Defaults**: class=4 (Desktop), cpu=3.0, ram=8192, net=0 (LAN).
  - **Invalid Strings:** Reject any input that is not 4 or 8 fields with Error 400.
- **Data Structure:** Update the Job struct to include:
  - int device_class, float cpu_freq, int ram_mb, int net_type.
  - bool is_default_cap (set to true if defaults were used).
- **Performance:** Ensure the parser remains **O(N)** to protect the accuracy of the **Time-to-First-Reply (TTFR)** measurement.

Summary: The system must be backward compatible but must prioritize 8-field telemetry to enable the slowness factor (θ_c) calculations.

You must implement two distinct high-precision timers. These timers operate at different stages of the request lifecycle.

1. Timer 1: The Initial Handshake Probe (Admission Stage)

This timer identifies the client's hardware capacity **before** a core is committed.

- **Logic:** Upon request arrival, the SISE. Engine (the server) sends an initial cryptographic handshake message (or a dummy ping) to the client and the **Resource Manager** starts the timer.
- **Trigger:** The timer **stops** when the client's response is received by the server.
- **Goal:** Calculate the Round Trip Time (RTT) or **Time-to-First-Reply (TTFR)** to determine the **Slowness Factor (θ_c)**.
- **System Action:**
    - **Normal Client:** Proceed to standard enqueuing.
    - **Weak Client:** Initiate the **Negotiation State Machine**. Propose a shift to a lighter model (e.g., VGG16 to HiNet).
    - **Agreement:** If the client accepts, establish a Dynamic Service Level Agreement is reached.
    - **Refusal:** If the client refuses the shift, do not reject. Instead, assign a **Priority Penalty ($\phi = 1$)** to move the request to the back of the **Final Sorted Queue**.

2. Timer 2: The ReLU Audit Hook (Execution Stage)

Since SNNI is **non-preemptive**, we cannot stop a "hostage core" once it starts. This timer gathers "ground-truth" data for future accuracy.

- **Logic:** When the **Dispatcher** starts an online inference job (J_{on}), the **Resource Manager** monitors the execution time.
- **Trigger:** The timer specifically captures the latency of the **first interactive protocol round** (e.g., the first ReLU layer).
- **Responsibility:**
    - **Resource Manager:** Acts as the stopwatch to record the actual duration.
    - **Profiler:** Receives the duration and calculates the actual θ_c.
- **Outcome:** The **Profiler** updates the permanent **Profiling Table (Π)** using the **Exponential Weighted Moving Average (EWMA)**.
- **Purpose:** This ensures that the **Admission Control** decisions (and VFT estimations) for all **future** requests become increasingly accurate.

Summary of Responsibilities:

Timer	Component	Stage	Purpose
Timer 1	Admission Control	Pre-Admission	**Decision Making:** Negotiation and Priority Penalty.
Timer 2	Resource Manager	Execution (J_{on})	**System Learning:** Database refinement for future requests.

Developer Action: Ensure Timer 1 happens at the TCP gateway level and Timer 2 is implemented as a non-blocking wrapper around the SNNI protocol function calls

Regarding **Q-2: TTFR Handshake Probe Implementation**, follow these concise instructions:

- **Timeline:** Implementation is **Mandatory for Phase 1**.
- **Reason:** Without measured TTFR, the system cannot calculate the **Asymmetry Factor (θ_c)**. This would treat all clients as symmetric ($\theta_c = 1.0$), which eliminates the primary scientific novelty of the paper (the "Hostage Core" effect).

- **Action (a) - Timer:** In the SISE gateway, wrap the **cryptographic mask distribution** in a high-precision timer (std::chrono::steady_clock). Start the timer when the server sends the masks (z) and stop it the moment the client's blinded input ($\hat{x}$) is received.
- **Action (b) - Computation:** Calculate the verified **Slowness Factor** as: $\theta_c = \frac{TTFR_{\text{actual}}}{TTFR_{\text{baseline}}}$. The baseline value must be retrieved from the **Profiling Table (Π)** based on the model and device class.
- **Action (c) - Logic Flow:** The resulting θ_c must be immediately available to **Admission Control** to perform the **Virtual Finish Time (VFT)** check.
- **Requirement:** Ensure the calculation happens **before** the request is enqueued. This allows the system to initiate model negotiation or apply the **Hostage Core Penalty** before resources are committed.

Regarding **Q-3: Negotiation State Machine**, follow these concise instructions:

- **Timeline:** Implementation is **Mandatory for Phase 1**.
- **Reason:** This is a core scientific contribution. It allows the system to actively manage the "Hostage Core" effect rather than just observing it.
- **Action (a) - Threshold:** If the verified (verified $\theta_c \geq 1.5$), trigger the negotiation.
- **Action (b) - Lookup:** Query profile.cfg for the n_{alt} mapping (e.g., AlexNet → HiNet).
- **Action (c) - Response Handling:** Implement an **asynchronous wait**. Give the client (e-g. **200 ms)** to respond to the SUGGESTED_MODEL proposal.
- **Action (d) - Penalty Logic:**
  - If the client accepts n_{alt}: Set $\phi = 0$ and update the model type.
  - If the client refuses or times out: Set **Priority Penalty** $\phi = 1$.
- **Action (e) - Propagation:** The ϕ flag must be stored in the **Job struct**. The **Online Scheduler (SA/DQN)** must use $\phi = 1$ as a weight to lower the job's priority in the Final Sorted Queue.
- **Message Format:** Use the format defined in C-24: REJECT_ASYMMETRIC:SUGGESTED_MODEL:<n_alt>:SUGGESTED_SLO:<vft_value>.

Summary: The full state machine is required. It ensures that resource-constrained clients either shift to efficient models or are deprioritized to protect the global utility metric Ψ.

Regarding **Q-4: 3-Tier SLO Filtering**, follow these concise instructions:

- **Timeline: Implement all three tiers now** (Phase 1).
- **Reason:** Tiers 1 and 2 act as early-exit "sanity checks." They prevent the server from wasting CPU and 35GB memory blocks on requests that are mathematically impossible to satisfy, protecting the global utility metric Ψ.
- **Sequential Logic:**
  1. **Tier 1 (Practicality):** Check if $D_{i,k} < e_{on}$. (Use baseline symmetric execution time).
  2. **Tier 2 (Static SLO-factor):** Check if $D_{i,k} <$ (factor $\times E_{i,c}$). Use the verified θ_c from the Handshake Probe (Q-2) to calculate $E_{i,c}$.
  3. **Tier 3 (Dynamic VFT):** Perform the existing VFT check as the final admission gate.
- **Error Codes:**
  - **Return 403** for **Tier 1 and Tier 2** rejections (Category: *Practicality/Parameter Error*).
  - **Return 404** for **Tier 3** rejections (Category: *System Congestion*).
- **Requirement:** Ensure Tier 1 and Tier 2 are checked **before** the VFT calculation to minimize server-side overhead for invalid requests.

Summary: Since the Handshake Probe (Q-2) is mandatory for Phase 1, you will have the data needed for Tier 2. Implement the full 3-tier sequence to ensure the system only admits "winnable" requests.

Per requirement **C-10** and **Q-5**, you must implement the expanded **21-column CSV trace**. This schema is mandatory for Phase 1 to ensure our data analysis scripts remain compatible. Regarding **Q-5: 21-Column CSV Trace**, follow these concise instructions:

- **Timeline:** Implementation is **Mandatory for Phase 1** (using Option A).
- **Reason:** Fixing the CSV schema now prevents breaking our automated post-hoc analysis scripts (Python/Matplotlib) later. A consistent 21-column format is required to prove the "Spatio-Temporal" and "Learning Loop" claims in the final paper.
- **Requirement:** Implement all **21 columns immediately**. For features not yet active (e.g., before the first negotiation is triggered), use **placeholder values (0.0 or -1)**.

The 21-Column Trace Schema

Ensure the Logger.cpp outputs the columns in this specific order:

KairosSNNI 21-Column CSV Trace Schema

Index	Column Name	Description
1	**Arrival_TS**	Epoch timestamp when the request reached the SISE gateway (t_{arr}).
2	**Start_TS**	Timestamp when the server began the first phase of work (J_{pre}).
3	**Finish_TS**	Timestamp when the final online phase (J_{on}) completed (t_{fin}).
4	**Model_Batch**	Identifier string (e.g., alexnet_8).
5	**Total_Wait_Time**	Total duration spent in queues ($TAT - E_{i,c}$).
6	**Cores_Assigned**	The number of physical cores (p_J) assigned by the scheduler.
7	**CPU_Load**	Average CPU utilization percentage during the job execution.
8	**Mem_GB**	Peak memory occupancy in Gigabytes during the online phase.
9	**Energy_uJ**	Total energy consumed by the job in Microjoules (RAPL/Slurm delta).
10	**Requested_SLO**	The relative due date ($D_{i,k}$) provided by the client.
11	**Actual_TAT**	Total end-to-end turnaround time ($Finish\_TS - Arrival\_TS$).
12	**SLO_Diff**	The margin of success/failure ($Requested\_SLO - Actual\_TAT$).
13	**SLO_Met**	Binary flag: 1 if $Actual\_TAT \leq Requested\_SLO$, else 0.
14	**Exit_Code**	Process exit status (0: Success, -9: Timeout, etc.).
15	**Request_ID**	Unique identifier for the specific input (i, k, j).
16	**T_Start_Pre**	Absolute timestamp when the Offline Preprocessing (J_{pre}) started.
17	**T_Fin_Pre**	Absolute timestamp when the Offline Preprocessing (J_{pre}) finished.
18	**T_Start_On**	Absolute timestamp when the Online Inference (J_{on}) started.
19	**Theta_Est**	Slowness factor verified during the **Initial Handshake Probe**.
20	**Theta_Act**	Ground-truth slowness factor measured via the **ReLU Audit Hook**.
21	**Penalty_Phi**	Priority Penalty flag: 1 if negotiation was refused, else 0.

Implementation Instructions:

- **Delimiter:** Use a semicolon (;) as the delimiter to maintain consistency with existing Logger.cpp logic.
- **Header:** The CSV file must include these 21 column names as the first row.

- **Placeholder Rule:** If a feature (like the ReLU Audit) is not yet active in your build, you **must still output the column** using 0.0 as a placeholder. This prevents "Index Out of Range" errors in the analysis scripts.
- **Precision:** Timestamps and durations should be logged in **milliseconds (ms)** with at least 3 decimal places for slowness factors (θ).

Developer Action:

1. Refactor Logger.hpp: Update the header string to include these 21 labels.
2. Update Logger::logJob: Ensure all 21 fields are populated.
3. Consistency Check: If Timer 2 (ReLU Audit) is not yet implemented, Theta_Act should be logged as 0.0. Once implemented, it will be used to show the convergence of the EWMA Learning Loop.

Summary: Do not wait for the features to be finished. Create the "bucket" (the CSV schema) now so that our research data remains structured and reproducible from day one.
This 21-column trace is our "Source of Truth" for the scientific paper. It allows us to calculate the **State-Dwell Time** ($T\_Start\_On - T\_Fin\_Pre$) and the **Success Metric (Ψ)** during post-hoc evaluation.

Referring to 1. Project Overview 1.1 Three-Layer Architecture

There is a minor error that amongst others, Online Scheduler is a part of Request Scheduler. And Online Scheduler implements the (SA/DQN) and other baselines for comparison. So, the correction is Online Scheduler (with SA/DQN and other baselines for comparison).

Category B: Architectural Clarifications

Regarding **Q-6: Malleable Core Sizing in SA/DQN**, follow these instructions:

- **Timeline:** Active malleable core sizing is **Mandatory for Phase 1**.
- **Reason:** Static core counts from profile.cfg are insufficient for top-tier research. The "twofold" scientific novelty depends on the scheduler choosing both the **sequence** and the **partition size** (p_J).
- **SA Implementation:**
  - Add a fourth perturbation dimension: adjust_core_count.
  - During the search, the SA must randomly try changing a job's core allocation (e.g., from 8 cores to 16 cores).
  - The SA cost function must evaluate if using more cores for a "heavy" job improves the global utility Ψ by clearing memory faster.
- **DQN Implementation:**
  - Add assigned_cores as an input feature in the state vector.
  - The DQN must learn the "Reward" of different core-count configurations for each model type.
- **Constraint:** The scheduler must ensure that at any time t, the sum of all assigned cores satisfies $\sum p_J \leq P$.

Summary: Do not use the threads field in profile.cfg as a fixed value. Use it as a **default starting point**. The SA and DQN agents must actively determine the optimal core count to maximize the success-to-latency ratio (Ψ).

Regarding **Q-7: DQN Hot-Reload Mechanism**, follow these instructions:

- **Timeline:** Implementation is **Optional for Phase 1**
- **Reason:** This enables the "Closed-Loop Learning" story for the paper. We need to demonstrate that the system can improve its policy based on real-time traffic without requiring a server

restart. **OR** The system can function using a pre-trained weight file. The "Learning Loop" can be demonstrated in a later phase. **Action:** Load weights once at startup for now.

- **Method: Option (a) File-Watch (Automatic Refresh).** This is the most robust method for a research artifact.
- **Logic for DQNScheduler:**
    1. **Monitor:** Check the mtime (last modified time) of the weight binary file (e.g., dqn_weights.bin) every 10 seconds.
    2. **Atomic Swap:** Load the new weights into a **temporary buffer** first. Once the file is fully read and verified (see C-20 for the hash/checksum check), perform a pointer swap to update the active model.
    3. **Safety:** A weight swap **must not** occur while the Online Scheduler is in the middle of a sortQueue() operation. The swap must happen between scheduling cycles.
- **Workflow Integration:**
    - The C++ SISE logs (s, a, r, s') tuples to the 1TB space.
    - The Python background process trains on these logs and saves a new .bin file.
    - The SISE detects the change, performs the hot-reload, and immediately begins using the refined policy.

Summary: Implement an automatic file-watch mechanism. This allows the system to adapt its scheduling priority and malleable core allocation in real-time as network conditions or client behaviors shift.

Regarding **Q-8: 100ms Continuous Memory Polling**, follow these instructions to provide the empirical data required for our "Spatio-Temporal Memory" analysis.

- **Timeline:** Implementation is **Mandatory for Phase 1**.
- **Reason:** We need a high-resolution time-series to visualize the "Memory Wall." Per-request snapshots cannot capture the cumulative memory pressure or the "sawtooth" pattern of the proactive buffer.

Technical Specification:

1. Log Format: Separate File (mem_trace.csv)

- **Decision:** Use a separate file.
- **Reasoning:** You cannot mix event-based data (Request logs) with time-series data (Memory logs) in a single CSV without breaking post-hoc analysis scripts.
- **Schema:** Timestamp_ms; Mem_Active_GB; Mem_Buffer_GB; Cores_In_Use; Total_System_Load.

2. Interval: Configurable (Default 100ms)

- **Decision:** The interval must be configurable in config.cfg via the parameter TELEMETRY_SAMPLE_RATE_MS.
- **Range:** Support 50ms to 1000ms.
- **Note:** Keep the default at **100ms** for our primary research experiments.

3. Thread Control: Mandatory but Toggleable

- **Decision:** The thread must be active by default.
- **Requirement:** Add a flag ENABLE_TIME_SERIES_LOGGING: true in config.cfg. This allows us to disable the thread for pure performance/latency benchmarking if the 36K log entries induce I/O jitter.

4. Implementation Logic (SystemMonitorThread):

- **Action:** Implement a non-blocking std::thread that sleeps for the configured interval.
- **State:** It must query the ResourceManager to get the split between **active job memory** and **proactive buffer memory**.

- **Storage:** Write directly to the **1TB Project Space** to ensure large log files do not exhaust the home directory quota.

Summary: Provide a dedicated time-series memory logger. This allows us to prove that **KairosSNNI** prevents memory saturation during multi-tenant bursts, which is the heart of our scientific contribution.

As per my understanding, I answer the following question but your suggestions are also welcome that how can we handle this situation to get better results.

Regarding **Q-9: Scheduler-Strict Core Allocation**, follow these instructions:

- **Timeline:** Implementation is **Mandatory for Phase 1**.
- **Decision:** Switch to **Strict Mode**. The system must never assign fewer cores than the count (p_J) determined by the **Online Scheduler**.
- **Preferred Fallback:** Use **Bypass (Non-blocking HoL Mitigation)**. If the required cores for the top job are unavailable, the **Dispatcher** must skip it and check the next job in the **Final Sorted Queue**.

Technical Reasoning:

1. **Protecting the Memory Wall:** In SNNI, a running job "locks" a massive preprocessed file (e.g., 35GB) in RAM. If you give a job 4 cores instead of 16, it takes much longer to finish. This keeps the 35GB file stuck in memory longer, blocking other requests and crashing the Ψ metric.
2. **Maintaining Scheduler Intelligence:** The **SA/DQN** algorithms chose a specific p_J to maximize SLO attainment. "Elastic shrinking" invalidates the scheduler's math and leads to certain SLO violations.
3. **HoL Mitigation:** By using **Bypass** instead of **Wait**, the system ensures that 24 cores do not sit idle. Smaller jobs that fit the current "resource gap" can jump ahead, maintaining high throughput.

Summary for Task List:

- **Modify ResourceManager.cpp:** Implement acquireCoresStrict(int p_target). Return false if not enough cores are free.
- **Modify Dispatcher.cpp:** Update the loop to iterate through the queue. If a job fails the core or memory check, **bypass** it and move to the next. Do not block the entire system.
- **Logging:** Record the number of "Bypass Events" in the trace to evaluate the effectiveness of this strategy.

13th Feb Answers

The following items are marked "Mandatory Phase 1" by the client but require access to the SHARK codebase for full implementation. Placeholder fields and infrastructure are in place.

2.1 Q-2: TTFR Handshake Protocol

Client Answer: Mandatory Phase 1. Two-timer approach:

- Timer 1 (Admission RTT): Measured at TCP connection time. Estimates theta_c from network delay.
- Timer 2 (ReLU Audit Hook): Executed during the first SHARK inference. Provides ground-truth theta_c.
- Current Status: theta_est = 1.0 (placeholder), theta_act = 0.0 (placeholder).
- Blocker: Timer 2 requires instrumentation inside SHARK benchmark binaries (ReLU Audit Hook callback).

Question for Client: Can you provide the SHARK benchmark source code or a callback interface specification for the ReLU Audit Hook? Without this, Timer 2 cannot be implemented. Timer 1 (RTTbased) can be approximated from TCP socket timing if acceptable as an interim solution

Ans:
Regarding the blockers for **Q-2: TTFR Handshake Protocol**, please follow these revised instructions. Note that **no internal source code for the SHARK backend can be provided.**

1. Timer 1: Admission Probe (TCP Socket Timing)

- **Approval:** Your suggestion to use **TCP socket timing** for the mask distribution and blinded-input exchange is **approved** as the final solution for Timer 1.
- **Action:** Measure the duration between the server's initial transmission of setup parameters and the arrival of the client's first response. This value will populate theta_est.

2. Timer 2: Execution Audit (The "Black-Box" Approach)

- **Resolution of Blocker:** You are **not** required to instrument the internal SHARK source code or provide a callback interface.
- **Revised Action:** Implement an **External Execution Wrapper** around the entire SHARK online inference process (J_{on}).
- **Logic:** Since SHARK is a synchronous, interactive protocol, any client-side slowness will manifest as a delay in the **total execution time** of the server-side process.
- **Implementation:**
    1. Capture a high-precision timestamp immediately before launching the SHARK inference binary/process.
    2. Capture a timestamp immediately upon the process termination.
    3. The duration Δt will serve as the ground-truth execution time.
    4. The Profiler shall use this total duration to calculate the final theta_act by comparing it to the profiled symmetric baseline.

3. Summary of Implementation for Phase 1

- **SHARK as a Black Box:** Treat the protocol engine as an atomic binary. Do not seek internal hooks.
- **Timer 1 (Network Level):** Captures initial RTT to set theta_est.
- **Timer 2 (Process Level):** Captures total synchronous execution time to set theta_act.
- **Result:** This allows you to complete the **EWMA feedback loop** and the **21-column trace** without needing internal SHARK source code.

Conclusion: Please proceed with the process-level timing for Timer 2. This satisfies the "Learning Loop" requirement while respecting the "Black-Box" nature of the cryptographic backend.

2.2 Q-3: Negotiation State Machine Client Answer: Mandatory Phase 1. Full state machine with theta threshold and alternative model lookup. • Threshold: theta_c >= 1.5 triggers negotiation (configurable via THETA_NEGOTIATION_THRESHOLD). • Timeout: 200ms client response timeout (configurable via NEGOTIATION_TIMEOUT_MS). • Penalty: phi = 1 if client refuses negotiation (already integrated into SA cost function). • Current Status: Config infrastructure ready. State machine logic requires Q-2 theta values. • Blocker: Without theta_c from TTFR, negotiation cannot be triggered.

Ans:
Regarding **Q-3: Negotiation State Machine**, follow these instructions to resolve the blocker:

1. Source of θ_c (Resolving the Blocker)

You do not need internal SHARK code to trigger negotiation. Use the **Timer 1 (Admission RTT)** result from the TCP socket timing (as approved in Q-2).

- **The Logic:** As soon as the initial mask/blinded-input exchange completes, the system calculates theta_est.
- **The Action:** Use this theta_est as the trigger for the Negotiation State Machine **before** the request is moved to the Global Queue.

2. Implementation Sequence

Follow this specific logic flow to ensure the state machine has the data it needs:

1. **Ingress:** Receive request string.
2. **Probe (Timer 1):** Execute the TCP-level handshake distribution. Calculate theta_est.
3. **Trigger:** if (theta_est >= THETA_NEGOTIATION_THRESHOLD) -> Start Negotiation.
4. **Negotiate:** Send SUGGESTED_MODEL to the client. Start the NEGOTIATION_TIMEOUT_MS timer (200ms).
5. **Finalize:**
   - If client accepts: Update model to n_{alt}, set penalty $\phi = 0$.
   - If client refuses or times out: Keep original model, set $\phi = 1$.
6. **Enqueue:** Place the finalized request into the Global Request Queue.

3. Handling placeholders

Continue using the infrastructure you have built. The "Blocker" is resolved by linking the output of your **Timer 1 (TCP logic)** directly to the input of the **Negotiation State Machine**.

Summary: Negotiation happens at the "Gatekeeper" stage using the network-level slowness factor. Timer 2 (Execution Audit) will be used later to refine this factor for future requests, but it is not required to start the initial negotiation.

Please proceed with integrating the TCP-based theta_est into the negotiation trigger.

2.3 Q-4 Tier 2: Static SLO Filter Client

Answer: Tier 2 uses asymmetric burst time: E = e_pre + (theta_c * e_on). • Tier 1 (Practicality: SLO >= e_on) is implemented and returns Error 403. • Tier 3 (Dynamic VFT with safety margin) is implemented and returns Error 404. • Tier 2 requires theta_c from Q-2 to compute E = e_pre + (theta_c * e_on). • Blocker: Same as Q-2 — theta_c values needed.

Ans:

Regarding **Q-4: Tier 2 Static SLO Filter**, follow these instructions to resolve the blocker:

1. **Source of Slowness Data:** Use the **theta_est** derived from **Timer 1 (TCP Socket Timing)** as the input for this filter. This allows the filter to function at the Admission stage before the job is enqueued.
2. **Calculation Logic:** Calculate the client-adjusted Burst Time using the formula: $E_{i,c} = e_{\text{pre}} + (\theta_{\text{est}} \cdot e_{\text{on}})$.
3. **Filter Logic:** Reject the request if the requested SLO ($D_{i,k}$) is smaller than the adjusted Burst Time multiplied by the configurable SLO-factor: If (D_{i,k} < SLO_factor * E_{i,c}) →**REJECT**.
4. **Error Code:** Consistent with Tier 1, return **Error 403** (REJECT_IMPRACTICAL_SLO).
5. **Sequential Order:** This check must happen **after** the Handshake Probe but **before** the Tier 3 (VFT) check.

Summary: The blocker is resolved by using the network-level estimate (theta_est). This ensures that even before the online inference begins, the system accounts for the "Hostage Core" effect when deciding if a request's SLO is realistic.

Please proceed with implementing the Tier 2 filter using the TCP-based theta_est.

3.1 Q-10: Asymmetric Burst Time Verification The contract specifies E_{i,c} = e_pre + (theta_c * e_on) for asymmetric burst time. Questions: • Is e_pre always unaffected by theta_c? (Pre-processing runs on

the server, not the client.) • Should theta_c apply only to the online inference phase (e_on)? • For theta_c < 1.0 (faster clients), should we allow E < e_pre + e_on? This could cause underestimation of deadline feasibility. • How should theta_c interact with the dynamic EWMA-smoothed estimates (dynamic_inf_ms)?

To the Developer,
Regarding **Q-10: Asymmetric Burst Time Verification**, please follow these instructions:

1. Is e_{pre} unaffected?

- **Answer:** Yes. The offline preprocessing phase (e_{pre}) is server-side only. It does not involve client interaction. Therefore, it is never affected by θ_c.

2. Should θ_c apply only to e_{on}?

- **Answer:** Yes. The online inference phase (e_{on}) is a synchronous interactive protocol. This is where the server core is "held hostage" by the client's speed. θ_c must only multiply the online duration.

3. Handling fast clients ($\theta_c < 1.0$):

- **Answer:** Do not allow $E < e_{\text{pre}} + e_{\text{on}}$. You must **floor** the slowness factor at 1.0 ($\theta_c = \max(1.0, \theta_c)$).
- **Reason:** Symmetric hardware is our best-case baseline. Underestimating execution time leads to aggressive admission and certain SLO violations during bursts.

4. Interaction with EWMA (dynamic_inf_ms):

- **Answer:** The variables are multiplicative.
- **The Formula:** $E_{i,c} = \text{dynamic\_pre\_ms} + (\theta_c \cdot \text{dynamic\_inf\_ms})$.
- **Logic:** The EWMA (dynamic_inf_ms) tracks the learned **server-side protocol baseline**. The θ_c factor scales that baseline to account for **client-side slowness**.

Summary:

- Always keep e_{pre} constant.
- Apply θ_c only to the online phase.
- Never let θ_c drop below 1.0.
- Combine θ_c with your learned EWMA values for the most accurate Burst Time ($E_{i,c}$).

3.2 Q-11: Psi Metric Script The contract references a post-hoc analysis metric Psi (Section 7.3). Questions: • Should Psi be computed in real-time or only as a post-processing analysis script? • What specific formula should be used? The contract mentions "composite scheduling quality" but does not provide the exact equation. • Should the Psi computation include energy metrics from Iteration 9? • What output format is expected? (CSV report, terminal output, dashboard metric?)

Ans:
Regarding **Q-11: Ψ Metric Implementation**, please follow these technical specifications for implementation and analysis:

1. The Exact Formula

The primary objective of the KairosSNNI orchestrator is to maximize the utility metric Ψ. Use the following equation:

[\Psi = \frac{\sum_{m \in I_\tau} \chi_m}{\sum_{m \in I_\tau} TAT_m}]

Variable Definitions:

- $\sum \chi_m$**:** The aggregate count of inputs that successfully met their Service Level Objective ($TAT_m \leq D_{i,k}$).

- $\sum TAT_m$**:** The total sum of end-to-end turnaround times (Total Sojourn Time) for all processed inputs.
- **Logic:** This represents the **density of success per unit of latency**. Maximizing this ratio forces the scheduler to minimize aggregate delay while specifically prioritizing "winnable" deadlines.

2. Computation Point: Dual-Tier Implementation

- **Real-time (Server-side):** The **Online Scheduler (SA/DQN)** must compute a running estimate of Ψ locally. This is required for the DQN reward signal and SA cost evaluations.
- **Post-hoc (Analysis Script):** The definitive Ψ value for the research paper must be calculated via an external Python script (analysis.py) using the raw timestamps from the 21-column CSV trace.

3. Energy Metrics Integration

- **Decision:** Do not include energy in the primary Ψ denominator.
- **Reasoning:** Ψ is a Quality-of-Service metric (Success vs. Latency).
- **Action:** Energy ($Energy_{uJ}$) is a secondary performance metric. The post-hoc script should report **Success-per-Joule** as a separate efficiency metric, but the primary Ψ remains time-based.

4. Output Format

The system must provide two types of outputs:

- **Terminal Output:** At the end of an experimental run, the server should print a summary: [SUMMARY] Total Inputs: X | Success Rate: Y% | Global Utility (Psi): Z
- **CSV Report:** The analysis.py script should generate a summary_report.csv containing Ψ, average TAT, tail latency ($P99$), and total energy consumption for each scheduling algorithm tested.

Summary:

- **Formula:** Success Count / Total Latency Sum.
- **Role:** Real-time for RL training; Post-hoc for paper results.
- **Scope:** Focus on timing; track energy as a separate efficiency index.

Please ensure the calculate_psi() function is unified across the SA and DQN modules to maintain mathematical consistency.

The utility metric Psi is defined as the ratio of satisfied SLOs to the aggregate turnaround times (TAT) for all processed jobs incurred by the system.

Formula:

Psi = [Sum of Chi_x] / [Sum of TAT_x]
(Summed over all inputs x in the set I_tau)

Definitions:

- **x:** An individual input.
- **I_tau:** The set of all individual inputs x processed within the time interval [0, tau].
- **Chi_x:** The SLO attainment flag. Chi_x = 1 if the turnaround time (TAT_x) is less than or equal to the relative due date (D_i,k), and 0 otherwise.
- **TAT_x:** The end-to-end turnaround time for each specific input x.

Primary Objective:

The primary objective of the server is to optimize the quality of service. The KairosSNNI scheduler seeks to identify a job execution sequence and spatial resource assignment that maximizes the optimization utility metric (Psi).
Maximizing Psi = the system simultaneously maximizes SLO attainment and minimizing the aggregate turnaround time TAT.

3.3 Q-12: Poisson Benchmarking Protocol The contract specifies Poisson-distributed request generation for benchmarking. Questions: • What lambda (arrival rate) values should be tested? (e.g., 0.1, 0.5, 1.0, 5.0 requests/sec) • Should the Poisson generator be built into the client or as a separate harness script? • What model mix should be used? (e.g., 70% simc1, 20% hinet, 10% alexnet) • Should SLO values be fixed or randomized per request? • How many total requests per test run? (e.g., 100, 500, 1000).

Ans:

Regarding Q-12: Poisson Benchmarking Protocol, please follow these technical specifications to ensure the benchmarking results provide the "Scientific Novelty" proof required for the paper.

1. Lambda (λ) Arrival Rates

To identify the "crossover point" where intelligent scheduling (SA/DQN) outperforms heuristics, we need to test across four load levels:

- 0.1 req/sec (Low Load): System under-utilization; tests baseline latency.
- 0.5 req/sec (Moderate Load): Initial contention for the proactive buffer and cores.
- 1.0 req/sec (High Load): Substantial memory pressure; high risk of Head-of-Line blocking.
- 2.0 req/sec (Stress Load): System saturation; tests the effectiveness of Admission Control rejections and the Priority Penalty (ϕ).

2. Benchmarking Harness

- Decision: The Poisson generator must be implemented as a separate Python harness script (e.g., benchmark_harness.py).
- Reason: This allows us to keep the client logic simple (execution only) while the harness manages the precise inter-arrival timings and captures aggregate statistics without being affected by client-side overhead.

3. Model Mix (Heterogeneous Workload)

To simulate a realistic multi-tenant environment, use the following distribution:

- 70% Lightweight: (SimC1, M1, or M2). Represents high-frequency, low-latency users.
- 20% Mid-tier: (HiNet or SimC2). Represents standard analytic tasks.
- 10% Heavyweight: (AlexNet or VGG16). Represents complex, memory-intensive "blockers."

4. SLO Values (Due Dates)

- Decision: Randomized per request.
- Formula: $D_{i,k} = E_{i,c} \times \text{Uniform}(1.5, 4.0)$.
- Reason: Fixed SLOs are too predictable. Randomized SLOs force the DQN and SA algorithms to handle "Urgency Diversity," where a later-arriving job may have a much tighter deadline than one currently at the head of the queue.

5. Test Run Volume

- Requirement: 1,000 requests per test run.
- Reason: This volume is necessary to ensure the EWMA Learning Loop in the Profiler converges and to provide statistically significant data for the Cumulative Distribution Function (CDF) plots used in the paper.

Summary for Developer Task List:

- Develop benchmark_harness.py: Must support λ as a command-line argument.
- Implement Model Sampling: Use a probability-weighted selector for the 70/20/10 mix.
- Log the Setup: The harness must record the λ and model-mix configuration at the top of the resulting CSV trace.

This protocol is designed to "stress" the memory-compute coupling. It will clearly show how KairosSNNI maintains a high Ψ by bypassing heavy jobs to save the 70% of lightweight users during high-load bursts.

3.4 Q-13: Phase 1 Scope Clarification General scope questions for Phase 1 delivery: • Is the current test suite (107 checks across 18 test categories) sufficient for Phase 1 validation? • Should we prepare a demonstration script that exercises all scheduler modes (FCFS/SJF/EDF/LST/SA/DQN)? • Are there specific performance benchmarks (e.g., throughput, latency percentiles) that must be met? • What is the target deployment environment? (Slurm cluster specs, node count, GPU availability)

Ans:

Regarding **Q-13: Phase 1 Scope Clarification**, please follow these final directives to ensure the Phase 1 delivery aligns with the scientific objectives of the **KairosSNNI** research paper.

1. Test Suite Sufficiency

The current suite of 107 checks across 18 categories is **sufficient for functional validation**, provided it includes the following **SNNI-specific safety tests**:

- **Mask-Burn Verification:** A test ensuring that if a model shift occurs during negotiation, the initial VGG-16 masks are deleted and fresh HiNet masks are loaded.
- **Non-Preemption Enforcement:** A test verifying that once a job is pinned to a core, its t_fin matches $t_{start} + E_{i,c}$without interruption.
- **VFT Rejection Accuracy:** A test verifying that Error 403 (Practicality) and Error 404 (Congestion) are returned correctly according to the 3-tier filter logic.

2. Mandatory Demonstration Script

You **must** prepare a "Scheduling Tournament" demonstration script (run_kairos_tournament.sh).

- **Functionality:** This script should automatically execute the **Poisson Benchmarking Protocol** (from Q-12) six times—once for each scheduler mode (**FCFS, SJF-Aging, EDF, LST, SA, DQN**).
- **Comparison:** It should use the analysis.py script to generate a summary table comparing the **Global Utility Metric (Ψ)** across all modes.
- **Vibe Check:** This demo must clearly show that SA and DQN achieve a higher Ψ by "packing" smaller M1 jobs while background VGG-16 jobs are in the queue.

3. Performance Benchmarks (Success Criteria)

To pass Phase 1, the system must meet these technical targets:

- **Stability:** Zero Out-of-Memory (OOM) crashes during a burst of 5+ concurrent VGG-16 requests (tests the **Triple Check** and **90% RAM Cap**).
- **Efficacy:** The SA and DQN algorithms must show a $\geq 20\%$ **improvement in** Ψ compared to the FCFS baseline under high load ($\lambda = 1.0$).
- **Trace Integrity:** The 21-column CSV must be 100% populated (no missing fields) and time-series memory logs must be generated at 100ms resolution.

4. Target Deployment Environment

The system is optimized for **Edge-Cloud Asymmetric Server Nodes**.

- **Compute:** 24 physical CPU cores (AMD Genoa or equivalent).
- **Memory:** 64 GB to 128 GB Shared RAM.
- **Storage: 1 TB High-Speed Project Space** (Mandatory for J_{pre} file staging).
- **Software:** Linux environment with Slurm support. The SISE should run as a persistent daemon, while clients connect via the TCP gateway.
- **Note on GPUs:** As this is an FSS-based protocol (SHARK), the focus is on **CPU-bound modular arithmetic**. GPU acceleration is out of scope for Phase 1.

Summary of Phase 1 Deliverables:

1. **Orchestrator Source Code:** Fully functional SISE with all 6+1 scheduler modes.
2. **Telemetry Trace:** High-fidelity 21-column request logs and 100ms memory logs.

3. **Benchmarking Harness:** Python scripts for Poisson request generation.
4. **Research Artifacts:** The README.md and analysis.py scripts for results reproducibility.

This scope focuses purely on the **Spatio-Temporal Coupling** and **Asymmetry** novelty. If the tournament script proves Ψ is maximized, the Phase 1 delivery is a success.

Let me explain again:

1. Admission Control & Asymmetry Probing (Q-2, Q-3, Q-4)

Technical Decision: The **Handshake Probe** must be the first interactive step upon connection. It verifies the client before resources are locked.

- **Handshake (Timer 1):** Use the TCP distribution of cryptographic masks (z). Start timer on send(z), stop on recv(x_hat). This calculates $TTFR_{actual}$ and sets theta_est.
- **Identification logic:**
- $\theta_c = \max(1.0, TTFR_{actual}/TTFR_{baseline})$.
- **Negotiation State Machine:** If
- $\theta_c \geq 1.5$, trigger negotiation **before** enqueuing.
  - Propose n_{alt}(lookup from profile.cfg).
  - **200ms Timeout:** If no reply, keep the original model and set **Priority Penalty** $\phi = 1$.
  - **The Mask-Burn Rule:** If a model shift is accepted (e.g., VGG16 →HiNet), you **must delete/invalidate** the VGG16 mask used for the probe and pull a fresh HiNet material file for the inference.
- **3-Tier SLO Filter:**
  - **Tier 1 (Practicality):** $D_{i,k} < e_{on}$(Baseline). Return **Error 403**.
  - **Tier 2 (Static SLO-Factor):** $D_{i,k} < (SLO\_factor \cdot E_{i,c})$. Return **Error 403**.
  - **Tier 3 (Dynamic VFT):** $VFT \cdot \sigma > D_{i,k}$. Return **Error 404**.

2. Burst Time & Performance Metrics (Q-10, Q-11)

Technical Decision: The Burst Time ($E_{i,c}$) must strictly follow the asymmetric coupling logic to protect the "Memory Wall."

- **Asymmetric Burst Time:**
- $E_{i,c} = e_{pre} + (\theta_c \cdot e_{on})$.
  - e_{pre} is constant (server-only). θ_c scales e_{on} (hostage core effect).
  - $\theta_c$is floored at 1.0 to prevent underestimation.
- **The Ψ (Psi) Utility Metric:**
  - **Formula:**
  - $\Psi = \frac{\sum_{m \in I_\tau} \chi_m}{\sum_{m \in I_\tau} TAT_m}$
  - χ_m**:** 1 if $TAT_m \leq D_{i,k}$, else 0.
  - **Goal:** Maximize Success Density per unit of Latency.
  - **Implementation:** OnlineScheduler uses a real-time estimator of Ψ; analysis.py calculates the definitive result from raw CSV timestamps.

3. Execution & Resource Management (Q-6, Q-9)

Technical Decision: The Dispatcher is a strict enforcer of the Scheduler's optimized plan.

- **Strict Core Allocation:** The Dispatcher **must not** shrink core counts. If a job is assigned $p_J = 16$ cores by the Online Scheduler, it only starts if 16 cores are free. If not, **Bypass** the job (HoL Mitigation) and check the next one in the sorted queue.

- **Malleability:** Core sizing ($1 \ldots P_{max}$) is a decision made by the **SA/DQN Scheduler** during the permutation search, not by the Resource Manager at runtime.
- **Non-Preemptive Lease:** Replace the 600s timeout with a dynamic lease:
- $T_{lease} = E_{i,c} \cdot 1.2$. This ensures the "Hostage Core" has enough time to finish synchronous rounds without being killed prematurely.

4. Evaluation & Benchmarking (Q-12, Q-13)

Technical Decision: Benchmarking must demonstrate the "Crossover Point" where AI-driven scheduling justifies its overhead.

- **Poisson Arrival ():** Test levels $0.1,0.5,1.0,2.0$ requests/sec.
- **Model Mix:** 70% Small (M1/SimC1), 20% Medium (HiNet), 10% Large (VGG16/AlexNet).
- **Due Dates:** Randomized using
- $D_{i,k} = E_{i,c} \times \text{Uniform}(1.5,4.0.$
- **Data Integrity:** The 21-column CSV trace is mandatory. It must capture the **State-Dwell Time** ($T\_Start\_On - T\_Fin\_Pre$) to prove the scientific novelty of our memory management.
- **Infrastructure:** Use the **1 TB Project Space** for all preprocessed file I/O to avoid home-directory quota exhaustion.

Summary Table of Mandatory Deliverables (Phase 1)

Module	Core Logic	Verification Method
Admission	8-field parser + 3-tier SLO filters	Error code matching (403/404)
Negotiation	Model shift (n_{alt}) + Mask Burning	Log check for file deletion upon shift
Profiler	EWMA + Welford's (σ)	Convergence of $\theta_c$over 1,000 requests
Scheduler	Malleable SA/DQN optimizing Ψ	Competitive analysis vs. FCFS/EDF
Dispatcher	Strict Core Pinning + Triple Check	Trace check for $p_{assigned} == p_{actual}$
Telemetry	21-column trace + 100ms Memory poll	Plotting the "Memory Wall" graph

Final Instruction: No internal SHARK code changes are permitted. Use external wrappers for the **ReLU Audit Hook** timing. All IP belongs to the research project and is subject to strict confidentiality.

To be clear: **Phase 1 includes the full implementation of the KairosSNNI Orchestration framework.**
Phase 1 is the **Initial Delivery of the complete, working logic.**
Here is the breakdown of what is included in Phase 1 and why it represents the "whole implementation":

1. What Phase 1 IS:

Phase 1 is the delivery of the **Secure Inference Service Engine (SISE)**. This includes:

- **The Full Algorithm Suite:** FCFS, SJF, EDF, LST, **Simulated Annealing**, and **DQN**.
- **The Complete Admission Logic:** All 3 SLO filters and the **Negotiation State Machine**.
- **The Asymmetry Probing:** Both **Timer 1** (Handshake) and **Timer 2** (ReLU Audit Wrapper).
- **The Resource Manager:** Core pinning, memory capping, and the memory (**1TB Project Space)** management.
- **The Data Trace:** The full **21-column CSV** and the **100ms memory log**.

2. What Phase 1 IS NOT (and why):

The only thing the developer is **not** doing is changing the **SHARK cryptographic code itself**.

- **Reason:** We have finalized that they will treat SHARK as a **"Black Box."**
- **How it works:** They wrap the SHARK binary in their code. This allows them to implement the "Whole Implementation" of the **Scheduler** without needing to be a cryptographer or having the internal SHARK source code.

3. Why the developer mentioned "Placeholders":

The developer was "short on information" and used placeholders (dummy values) because they weren't sure how to measure client speed (θ_c) or how to suggest models (n_{alt}).

Now that you have provided the following, the placeholders are removed:

- **C-02:** They have the 8-field wire protocol.
- **C-04:** They have the specific Model Mapping (VGG16 →AlexNet).
- **C-06:** They have the Burst Time formula with θ_c.
- **C-32:** They have the Standard Deviation logic (Welford's).

4. Summary of the Phase 1 Goal

Phase 1 is meant to produce a **Reproducible Research Artifact**.

By the end of Phase 1, you should be able to:

1. Run the server and the benchmark harness.
2. See the system identify a "Weak Client."
3. See the system negotiate or apply a penalty.
4. See the **SA/DQN** reorder the queue.
5. Generate the CSV files that prove your scientific novelty.

Conclusion:

Phase 1 **is** the full implementation of the **KairosSNNI system** as we have defined it. It includes the real, working code.

Final deliverable:

Phase 1 requires the **complete functional implementation** of the SISE as specified in the 32 technical decisions. There should be no placeholders in the final Phase 1 delivery; all logic, from asymmetry probing to SA/DQN scheduling, must be operational.